

Vysoká škola ekonomická v Praze
Fakulta informatiky a statistiky



Návrh a testování prostředí pro AI agenty v softwarovém vývoji

BAKALÁŘSKÁ PRÁCE

Studijní program: Aplikovaná informatika

Autor: Thanh An Nguyen

Vedoucí práce: Ing. Jiří Korčák

Praha, květen 2026

Prohlášení

Prohlašuji, že jsem bakalářskou práci zpracoval samostatně a že jsem uvedl všechny použité prameny a literaturu, ze kterých jsem čerpal.

Potvrzuji, že při přípravě předložené práce **byly použity** nástroje umělé inteligence. Konkrétně byly použity těmito způsoby:

- úprava a rozšíření textových částí,
- vyhledávání a shrnutí literatury,
- diagnostická analýza výsledků experimentu,
- generování částí zdrojových kódů měřicí infrastruktury.

Každé použití některého ze způsobů je samostatně dokumentováno v příloze A, kde je uvedeno podrobnější vysvětlení, který nástroj byl ve které části práce konkrétně použit, a je zvýrazněn vlastní přínos.

Potvrzuji, že:

- má práce je originální a byla zpracována v souladu s etickými standardy, zejména s Etickým kodexem Vysoké školy ekonomické v Praze, že jsou v práci citovány všechny použité informační zdroje a že generovaný obsah byl mnou ověřen.
- přijímám plnou odpovědnost za celý obsah závěrečné práce.

V Praze dne 10. května 2026

Thanh An Nguyen

Poděkování

Děkuji vedoucímu práce Ing. Jiřímu Korčákovi za odborné vedení, trpělivost a věcné připomínky, které práci v průběhu jejího vzniku významně posunuly. Poděkování patří také mé rodině a blízkým za podporu během celého studia.

Abstrakt

AI coding agenti dokáží samostatně generovat kód, testy i vývojové artefakty, ale jejich praktická použitelnost nezávisí jen na tom, zda výsledný program projde testy. Pro nasazení ve vývoji je důležité také to, jak agent pracoval, zda zanechal auditovatelnou stopu a jakou kvalitu má výsledný kód. Současné benchmarky tyto dimenze typicky nezachycují společně a chybí postup, jak podle nich systematicky navrhovat instrukce. Práce proto navrhuje sadu metrik pokrývajících proces, kvalitu produktu a zdroje a ukazuje, jak ji použít k iterativnímu návrhu instrukcí. Proveditelnost postupu ověřuje případová studie systému upomínek faktur. Agent opakovaně implementuje stejnou specifikaci, výsledky jsou vyhodnoceny navrženými metrikami a instrukce jsou mezi běhy upravovány podle diagnostiky selhání. Následné ablace zkoumají, které složky instrukcí mají měřitelný přínos. Výsledky ukazují, že metriky dokáží odlišit funkčně úspěšný výstup od výstupu vzniklého slabým nebo neauditovatelným procesem. Iterativní úpravy instrukcí vedly ke zlepšení, ale toto zlepšení nebylo plynulé napříč běhy. Některé změny způsobily regresi a stejné instrukce vedly v různých bězích k odlišnému dodržení pracovního postupu. Opakovaným vzorcem úspěšných úprav byl posun od obecného pravidla ke konkrétnímu příkazu a nakonec k verifikačnímu kroku. Ablace ukázaly, že verifikační kroky nejsou redundantní. Odebrání části kódových konvencí deterministické metriky téměř nezhoršilo, ale zhoršilo designovou kvalitu hodnocenou LLM-as-judge. Přínosem práce je ověření proveditelnosti sady metrik a iterativního postupu návrhu instrukcí na jednom případě. Konkrétní naměřené hodnoty platí pro daný model, nástroj a projekt. Přenositelný je postup a sada metrik. Otevřenou otázkou zůstává, zda vzorec operacionalizace platí i mimo programování.

Klíčová slova

AI coding agent, AGENTS.md, metriky kvality software, iterativní návrh, ablační studie

Abstract

AI coding agents can generate code, tests, and development artifacts autonomously, but their practical usefulness depends on more than whether the resulting program passes tests. In software development, it also matters how the agent worked, whether it left an auditable trace, and what quality the resulting code has. Existing benchmarks typically do not capture these dimensions together, and there is no established process for designing instructions against such measurements. This thesis proposes a metric suite covering process, product quality, and resources, and shows how it can guide iterative instruction design. The feasibility of the process is evaluated on a case study of a billing reminder system. The agent repeatedly implements the same specification, each run is evaluated using the proposed metrics, and the instructions are revised according to a diagnosis of observed failures. Subsequent ablations examine which parts of the instructions have a measurable contribution. The results show that the metrics can distinguish a functionally successful output from one produced by a weak or non-auditable process. Iterative instruction changes improved the agent’s behavior, but the improvement was not steady across runs. Some changes caused regressions, and identical instructions led to different levels of process adherence across runs. A recurring pattern in successful changes was a shift from a general rule to a specific command and then to a verification step. Ablations showed that verification steps were not redundant. Removing part of the code-convention section had little effect on automated metrics, but reduced design quality as assessed by an LLM-as-judge. The contribution is a feasibility evaluation of the metric suite and the iterative instruction-design process on one case. The specific measured values are bound to the model, tool, and project used in the case study. What transfers is the process and the metric suite. An open question is whether the observed operationalization pattern also holds outside programming.

Keywords

AI coding agent, AGENTS.md, software quality metrics, iterative design, ablation study

Obsah

Úvod	11
1 Vymezení problému a cílů práce	12
1.1 Motivace	12
1.2 Cíle práce	13
1.3 Rozsah práce	13
2 Teoretická východiska	14
2.1 Kvalita software a její měření	14
2.1.1 Procesní kvalita	15
2.1.2 Produktová kvalita	16
2.1.3 Zdrojová dimenze	18
2.2 AI coding agenti	19
2.2.1 Základní pojmy	19
2.2.2 Jak agenti mění softwarové inženýrství	20
2.2.3 Hodnocení schopností agentů	21
2.2.4 Instrukce jako nezávislá proměnná	22
3 Metodika	25
3.1 Výzkumný přístup	25
3.1.1 Volba výzkumné strategie	25
3.1.2 Případ a jednotky analýzy	26
3.1.3 Fixní proměnné	29
3.2 Sada metrik	30
3.2.1 Procesní metriky (P1–P8)	31
3.2.2 Produktové metriky (Q1–Q8)	32
3.2.3 Metriky zdrojů (E1–E3)	34
3.3 Procedura jedné iterace	35
3.3.1 Spuštění, záznam a vyhodnocení běhu	35
3.3.2 Diagnostika a úprava instrukcí	36
3.4 Přehledová tabulka	37
3.5 Omezení a validita	39
4 Praktická část	40
4.1 Konstrukce výchozích instrukcí	40
4.2 Pilotní fáze	42
4.2.1 Pilot-r1: výchozí běh	42
4.2.2 Pilot-r2	44
4.2.3 Pilot-r3	46
4.2.4 Pilot-r4	47

4.2.5	Pilot-r5: verifikace kontraktu	49
4.3	Ablace	50
4.3.1	Výběr složek pro ablaci	51
4.3.2	Ablace A: bez verifikačních kroků	52
4.3.3	Ablace B: bez Package Quality	54
4.4	Souhrnné výsledky	56
5	Vyhodnocení a diskuse	58
5.1	Interpretace výsledků	58
5.1.1	Cíl 1: Sada metrik	58
5.1.2	Cíl 2: Iterativní postup	62
5.1.3	Cíl 3: Ablace	64
5.2	Porovnání s literaturou	65
5.2.1	Srovnání sady metrik	65
5.2.2	Vliv instrukcí na chování agenta	65
5.3	Omezení a jejich dopad	66
5.4	Doporučení pro praxi	68
5.5	Náměty pro další výzkum	69
	Závěr	72
	Literatura	74
	A Využití nástrojů umělé inteligence	81
	B Doplnující materiály k případové studii	82

Seznam obrázků

1	Stavový automat systému upomínek	27
2	Iterativní cyklus pilotní fáze.	35
3	Vizuální diff změn AGENTS.md mezi pilot-r1 a pilot-r2	44
4	Vizuální diff změn AGENTS.md mezi pilot-r2 a pilot-r3	46
5	Vizuální diff změn AGENTS.md mezi pilot-r3 a pilot-r4	47
6	Vizuální diff změn AGENTS.md mezi pilot-r3 a pilot-r5	49
7	Změny AGENTS.md pro Ablaci A	51
8	Vizuální diff AGENTS.md mezi pilot-r3 a ablací B.	52
9	Míra splnění exit kritérií procesní metriky (P1–P8) napříč devíti běhy.	59
10	Míra splnění exit kritérií produktové metriky (Q1–Q8) napříč devíti běhy.	61
11	Cena běhu vs. počet splněných deterministických exit kritérií	62
12	Spektrum operacionalizace instrukcí.	63

Seznam tabulek

1	Procesní metriky dodržení pravidel P1–P5	32
2	Judge-based procesní metriky P6–P8	32
3	Metriky funkční korektnosti Q1–Q2	33
4	Metriky kvality testů Q3–Q4	33
5	Metriky kvality kódu Q5–Q8	34
6	Metriky zdrojů E1–E3	35
7	Záznamy pořízené pro jeden běh a jednu iteraci	36
8	Diagnostické rámce a otázky, které kladou	37
9	Mapování klasifikace příčiny na diagnostický rámec a typ opravy	37
10	Sada metrik podle kódu, měřené vlastnosti, nástroje a exit kritéria	38
11	Mapování sekcí výchozího AGENTS.md na metriky	40
12	Pilot-r1: naměřené metriky	42
13	Pilot-r1: nálezy a jejich příčiny	43
14	Pilot-r2: vývoj sledovaných metrik oproti r1	44
15	Pilot-r2: nálezy a jejich příčiny	45
16	Pilot-r3: metriky se změnou oproti r2	46
17	Pilot-r4: metriky se změnou oproti r3	48
18	Pilot-r4: nálezy a jejich příčiny	48
19	Pilot-r5: metriky se změnou oproti r3	49
20	Pilot-r5: nálezy a jejich příčiny	50
21	Ablace A: srovnání s r3 (bez verifikačních kroků)	53
22	Ablace B: srovnání s r3 (bez sekce Package Quality)	55
23	Souhrnné výsledky všech běhů (pilot + ablance)	57
24	Doplňující materiály k případové studii	82

Seznam použitých zkratek

E1 vstup/výstup/cache tokeny

E2 trvání

E3 počet kompakcí

P1 issues before code

P2 branch per issue

P3 test-first commits

P4 PRs linked to issues

P5 no existing test modifications

P6 kvalita commit messages

P7 kvalita issue descriptions

P8 kvalita PR descriptions

Q1 API contract match

Q2 úspěšnost referenčních testů

Q3 mutation score

Q4 AC coverage

Q5 lint warnings

Q6 typecheck errors

Q7 složitost kódu

Q8 design quality

Úvod

AI coding agenti dokáží autonomně implementovat funkcionalitu, psát testy, spravovat verzovací historii a komunikovat prostřednictvím issues a pull requestů. S rostoucí autonomií těchto nástrojů roste i potřeba systematicky hodnotit kvalitu jejich práce. Současné benchmarky hodnotí izolované aspekty práce agenta, především zda vygenerovaný kód splní zadání a projde testy. Holistický pohled, který by zachytil pracovní postup agenta, kvalitu produktu a zdroje společně, však chybí. Přitom i funkčně správné řešení může být v praxi odmítnuto kvůli nedostatečným procesním konvencím nebo udržitelnosti kódu, například pokud by neprošlo code review.

Kvalitu práce agenta výrazně ovlivňují instrukce, které dostane. V praxi se etabloují soubory s instrukcemi jako `AGENTS.md` nebo `CLAUDE.md`, které definují pracovní postup, konvence a omezení projektu. Tyto soubory se stávají standardním rozhraním mezi člověkem a agentem nejen v softwarovém vývoji, ale i v univerzálních agentních nástrojích. Jak instrukce systematicky navrhovat a jak měřit, zda agent dodržuje požadované praktiky, zůstává otevřenou otázkou.

Tato práce navrhuje sadu metrik pokrývajících pracovní postup agenta, kvalitu produktu a zdroje spotřebované při práci agenta. V případové studii agent opakovaně implementuje systém upomínek faktur ze specifikace, výsledky jsou měřeny navrženými metrikami a instrukce jsou mezi iteracemi upravovány podle diagnostiky selhání. Ablace dále zkoumají, které složky instrukcí přispívají k měřenému chování a které jsou redundantní.

Práce nejprve vymezuje problém a cíle, poté shrnuje teoretická východiska z oblasti softwarového inženýrství, AI agentů a měření kvality. Na jejich základě navrhuje metodiku, kterou ověřuje na případové studii. Kapitola Vyhodnocení a diskuse interpretuje výsledky ve vztahu k cílům, porovnává je s existujícím výzkumem a formuluje doporučení pro návrh instrukcí.

1. Vymezení problému a cílů práce

1.1 Motivace

AI coding agenti, tedy autonomní systémy schopné samostatně psát kód, spravovat verzovací historii a interagovat s vývojovým prostředím, se v posledních letech staly rozšířeným nástrojem každodenního vývoje softwaru. Slibují výrazné zrychlení práce, protože implementace, která vývojáři zabere hodiny, je teoreticky proveditelná v řádu minut. Randomizovaná studie METR (2025) na 246 úlohách ale ukazuje, že tento příslib se nenaplnuje automaticky. Vývojáři s AI nástroji byli ve skutečnosti o 19 % pomalejší.

Současné benchmarky jako SWE-bench (Jimenez et al., 2024) hodnotí izolované aspekty práce agenta, především funkční korektnost. Tři empirické studie ale shodně ukazují, že samotné binární hodnocení pro praktickou použitelnost nestačí. METR (2026)¹ doložila, že polovina patchů, které prošly automatickými testy, by nebyla přijata při code review. B. Li et al. (2026) na 1 210 sloučených pull requestech zjistili kvalitativní nedostatky i v přijatém kódu. Ehsani et al. (2026) pak z 33 tisíc agentních pull requestů identifikovali, že vedle funkčních chyb tvoří významný podíl odmítnutí i procesní selhání. Každá studie pojmenovává jinou stranu téhož problému. Praktická použitelnost agentního výstupu zahrnuje i review přijatelnost, udržitelnost a auditovatelnost postupu. Holistický pohled, který by tyto dimenze zachytil společně, dosud chybí.

Výsledky agenta nejsou dány jen schopnostmi modelu samotného. Lze je ovlivnit buď trénováním modelu, nebo instrukcemi v kontextovém okně. Tato práce se zaměřuje na druhou možnost. S rostoucí velikostí kontextových oken a zkracujícími se cykly mezi generacemi modelů jsou instrukce prakticky dostupnější než fine-tuning, protože je lze upravovat bez trénovacích dat, výpočetní kapacity a investice vázané na konkrétní verzi modelu. Shin et al. (2025) ukazují, že iterativní zpřesňování instrukcí dosahuje srovnatelných výsledků s fine-tuned modely na úlohách s kódem.

Empirie o účinnosti instrukcí je ale nejednoznačná. Lulla et al. (2026) zjistili, že přítomnost souboru s instrukcemi je spojena se zkrácením doby běhu o 28,6 %, zatímco Gloaguen et al. (2026) upozorňují, že generické instrukce přinášejí marginální zlepšení při vyšších nákladech. Které složky instrukcí k měřenému chování skutečně přispívají a které jen zabírají kontextové okno, dosud nebylo izolováno. Jak instrukce systematicky navrhovat a jak měřit, zda agent dodržuje požadované chování, zůstává proto otevřenou otázkou.

¹Jiná studie téže organizace než (METR, 2025). Zde jde o analýzu SWE-bench pull requestů, ne randomizovaný experiment.

1.2 Cíle práce

1. Na základě analýzy existujících standardů kvality softwaru a současných benchmarků pro AI agenty navrhnout sadu metrik pokrývající proces, kvalitu produktu a zdroje, a tím zachytit dimenze, které stávající benchmarky neměří.
2. Na případové studii ověřit proveditelnost iterativního postupu návrhu instrukcí řízeného těmito metrikami a vyhodnotit, zda a jak vede k měřitelným změnám v chování agenta podle navržených metrik.
3. Z instrukcí vytvořených v cíli 2 prozkoumat ablacemi, které složky přispívají k měřenému chování agenta a které jsou redundantní.

1.3 Rozsah práce

Cíle práce jsou naplňovány prostřednictvím případové studie systému upomínek faktur. Existující studie hodnotí agenty na velkém počtu krátkých úloh. Tato práce volí opačný přístup, tedy jeden projekt do hloubky, s opakovanými běhy a podrobnou analýzou každého z nich.

Práce se nezabývá trénováním modelů, protože cílem je zkoumat vliv instrukcí, ne schopnosti konkrétního modelu. Nesrovnává různé modely ani programovací jazyky, protože experimentální design se snaží držet ostatní podmínky co nejstabilnější a měnit především instrukce. Neporovnává agenta s lidským vývojářem, protože měří zlepšení mezi iteracemi, ne absolutní výkon. Navržená sada metrik a iterativní postup jsou koncipovány jako přenositelné výstupy práce. Konkrétní instrukce a naměřené hodnoty platí pro tuto případovou studii a slouží k ověření proveditelnosti, nikoli jako obecně platný výsledek pro všechny modely a projekty. Zdůvodnění volby projektu a podrobnosti experimentálního designu popisuje kapitola 3.

2. Teoretická východiska

Tato kapitola poskytuje teoretický základ pro sadu metrik a postup případové studie zaměřené na návrh a vyhodnocení instrukcí. První sekce se věnuje kvalitě software z pohledu softwarového inženýrství, tedy proč je její zajištění obtížné, jaké dimenze má, jakými praktikami se zajišťuje a jak se měří. Druhá sekce zavádí AI coding agenty: co jsou, jak se s nimi mění softwarové inženýrství, jak se jejich výstupy dnes hodnotí a jak se jejich chování řídí instrukcemi.

2.1 Kvalita software a její měření

Software je už ze své povahy složitý, a tato složitost má dva odlišné zdroje. Část pochází z povahy řešeného problému, tedy z domény, ve kterých software pracuje, jsou samy o sobě komplexní (mají mnoho stavů, výjimek a vzájemných závislostí), a tato složitost se nevyhnutelně přenáší do systému, který je má modelovat. Druhá část přichází z nástrojů a technologií, kterými software stavíme, například programovací jazyky, frameworky, build systémy a knihovny vnášejí vlastní vrstvu složitosti, která s podstatou úlohy přímo nesouvisí. Brooks (1987) tyto dva zdroje pojmenoval jako *esenciální* a *akcidentální* složitost a argumentuje, že zatímco akcidentální vrstvu lze snižovat lepšími nástroji, esenciální zůstává a je třeba ji systematicky zvládat.

Aby vývojáři tuto kombinaci složitostí udrželi pod kontrolou, opírá se obor o soubor zavedených praktik (testování, revize kódu, správa verzí, systematická diagnostika chyb), které tvoří jeho metodickou páteř (IEEE Computer Society, 2024). Samotné dodržování praktik ale kvalitu nezaručuje, a to ze dvou důvodů. Software, který se reálně používá, musí být průběžně upravován, jinak přestává odpovídat svému účelu (Lehman, 1980), a s každou úpravou roste jeho vnitřní složitost. Postupy vhodné v rané fázi vývoje proto nemusí být dostatečné později. Praktiky se navíc vyvíjejí společně s oborem, takže to, co bylo považováno za standard před deseti lety, dnes nemusí stačit. Kvalitu je proto třeba průběžně ověřovat měřeními, ne jen předpokládat z deklarace, že se nějaká praktika dodržuje.

Měření ale nemůže stát na jediné metrice ani na jediném typu evidence. Fenton and Bieman (2014) rozlišují tři kategorie měřitelných entit:

- *proces*, tedy aktivity a postupy, kterými produkt vzniká,
- *produkt*, tedy výstupy vývoje (implementace, testy, dokumentace),
- *zdroje*, tedy vstupy nutné k vzniku produktu (čas, úsilí, lidé, výpočetní prostředky).

Každá entita nese jiný typ evidence o kvalitě. Dodržení procesu nezaručuje funkční výsledek, funkčně správný produkt může vzniknout i chaotickým procesem a řešení může být z hlediska spotřebovaných zdrojů zbytečně nákladné. Ucelený obraz proto vzniká až jejich kombinací.

Následující tři podsekce rozebírají každou dimenzi zvlášť, konkrétně její konceptuální vymezení

a způsoby, jimiž ji literatura operacionalizuje. Hloubka zpracování se mezi nimi liší. Procesní stránka stojí na rodině etablovaných praktik vývoje software (sekce 2.1.1), produktová má dnes zralý standardizovaný rámec (sekce 2.1.2) a zdrojová přes nepřímé ukazatele spotřeby zdrojů (sekce 2.1.3).

2.1.1 Procesní kvalita

Procesní kvalitou se rozumí vlastnosti aktivit a postupů, kterými produkt vzniká. Fenton and Bieman (2014) mezi procesní atributy řadí zejména pracnost, trvání, shodu s definovaným postupem a stabilitu výstupu, tedy vlastnosti, které lze pozorovat na chování vývojáře, ne na hotovém kódu.

Stejný kód může vzniknout různými způsoby. Některé jsou auditovatelné a snadno opakovatelné, jiné stojí na jednorázových zkratkách a jejich stopa je pro tým obtížně dohledatelná. Verzovaný vývoj (typicky systémem Git, doplněným o platformové funkce jako issues a pull requesty na GitHubu nebo GitLabu) přitom po sobě zanechává řadu pozorovatelných artefaktů, jako jsou commity, větve, pull requesty a záznamy z CI.

Z těchto artefaktů lze průběh práce zpětně zrekonstruovat, a právě z nich, ne z deklarace o tom, jaký postup se údajně dodržuje, je třeba procesní kvalitu měřit. Jak velký rozdíl bývá mezi tím, co lidé o svém procesu říkají, a tím, co skutečně dělají, ukázali Beller et al. (2019) v rozsáhlé field study sledující 2 443 vývojářů v IDE po dobu 2,5 roku. Polovina z nich systematicky netestuje, test-driven development není rozšířený ani v komunitě, která se k němu hlásí, a čas skutečně strávený testováním je výrazně nižší než ten, který si vývojáři sami přisuzují.

Issue-driven vývoj a větvení

Změna v projektu obvykle začíná *issue*, tedy záznamem v issue trackeru, který popisuje, co je třeba udělat a jak poznat, že je úkol hotov. Issue plní dvě role současně. Dává implementaci jasné zadání ještě předtím, než vznikne kód, a zároveň zůstává trvalým záznamem, k němuž lze později dohledat související změny. Velkoplošná empirická studie Bissyandé et al. (2013) na stovkách tisíc projektů na GitHubu doložila, že issue trackery jsou v open source vývoji rozšířenou a aktivně používanou součástí projektové infrastruktury.

K issue se typicky váže vlastní vývojová větev, do které se implementace zapisuje. Práce na jedné změně tak zůstává izolovaná od ostatních, paralelní vývoj se vzájemně neruší a code review může posuzovat rozsah změny ohraničeně.

Malé commity a konvence commitů

Malé commity, kde každá změna odpovídá jednomu logickému kroku, usnadňují zpětnou analýzu chyb i code review (Humble & Farley, 2010). Kromě malých a častých commitů se v praxi rozšířila i konvence formátu commit zprávy: prefix v podobě **feat:**, **test:** nebo **fix:** signalizuje typ změny ještě před přečtením celé zprávy a zkracuje čas potřebný k orientaci v historii. Malé commity a konzistentní formát se ukazují jako nutná podmínka toho, aby commity nesly užitečnou informaci pro lidského i automatizovaného čtenáře.

Test-driven development a stabilita testů

Testování v průběhu implementace může mít různé pořadí, a zvolené pořadí mění roli, kterou test ve vývoji hraje. *Test-driven development* obrací nejběžnější *test-after* pořadí. Nejprve se napíše test definující očekávané chování, teprve pak implementace, která test splní (Beck, 2000). Tím se test stává specifikací požadavku, nikoli zpětnou kontrolou napsaného kódu, a vývojář je nucen vyjádřit záměr ještě před tím, než ho začne realizovat. Z této role plyne i praxe, že existující testy se v rámci implementace neupravují, jinak ztrácejí svou specifikační funkci. Meta-analýza 27 studií ukazuje, že TDD má pozitivní efekt na kvalitu kódu měřenou hustotou defektů, přičemž efekt je výraznější v průmyslových projektech než v akademických experimentech (Rafique & Mišić, 2013).

Code review a traceability pull requestů

Začleňování změn do hlavního kódu strukturuje code review, dnes ve své *modern code review* podobě integrované do pull requestů (Bacchelli & Bird, 2013). Kontrola probíhá průběžně nad menšími změnami, ne jako samostatná inspekce celé implementace. Aby review plnilo roli auditovatelného mostu mezi zadáním a kódem, je potřeba propojit pull request zpět na issue, ze kterého změna vznikla, typicky odkazem v těle PR (**Closes #N**) (Gotel & Finkelstein, 1994), a doplnit popis, který reviewerovi sděluje co a proč se měnilo. Tato traceability je předpokladem toho, aby bylo možné kdykoli zpětně rekonstruovat, proč byla změna provedena a jak bylo ověřeno, že odpovídá zadání.

2.1.2 Produktová kvalita

Zatímco procesní stránka popsaná v předchozí podsekcí sleduje, jak změna vznikla, produktová stránka se obrací k vlastnostem samotných výstupů vývoje. Produktovou kvalitou se rozumí vlastnosti zdrojového kódu, testů a dokumentace. Tyto vlastnosti nelze redukovat na jediné měřítko, protože různí stakeholdeři kladou důraz na jiné aspekty. Koncoví uživatelé a zadavatelé sledují především chování za běhu, tedy zda software dělá to, co se od něj očekává a zvládá okrajové případy. Vývojáři naopak posuzují čitelnost a modifikovatelnost kódu, se kterým

budou dál pracovat (Kitchenham & Pfleeger, 1996). Tyto pohledy jsou navíc vzájemně nezávislé. Funkčně správný software může být obtížně udržitelný, a naopak elegantně napsaný kód nemusí dělat to, co má.

Standard ISO/IEC 25010:2023 poskytuje standardizovaný rámec pro tuto vícevrstvou povahu kvality produktu (ISO/IEC, 2023). Model rozkládá kvalitu produktu do osmi charakteristik, mezi něž patří funkční vhodnost, výkonnostní efektivita, kompatibilita, použitelnost, spolehlivost, bezpečnost, udržitelnost a přenositelnost. Tyto charakteristiky lze posuzovat odděleně. Pro tuto práci jsou nejrelevantnější dvě. *Funkční vhodnost* vyjadřuje, do jaké míry software poskytuje funkce naplňující stanovené potřeby, a v praxi se zajišťuje testováním. *Udržitelnost* vyjadřuje, s jakou účinností lze software dále upravovat při opravách, zlepšeních nebo přizpůsobování změnám, a zajišťuje se především statickou analýzou. Obě deterministické vrstvy doplňuje lidské peer review pro aspekty, které pravidly nelze vyjádřit. Následující odstavce postupně popisují, jak se každá z těchto vrstev používá.

Funkční vhodnost a testování

Funkční vhodnost se zajišťuje testováním, tedy systematickou kontrolou chování implementace vůči očekávání. Testovací přístupy se podle toho, na čem staví, dělí na dva základní typy: *black-box* (funkcionální) testování pracuje přes veřejné rozhraní a ověřuje pozorovatelné výstupy nezávisle na vnitřní struktuře implementace, zatímco *white-box* (strukturální) testování vychází ze znalosti kódu a cíleně pokrývá jeho cesty (Ammann & Offutt, 2016).

V rámci *black-box* přístupu se ustavila praxe *behavior-driven testing*, kde testy explicitně formalizují očekávané chování jako specifikaci. Zdrojem této specifikace jsou typicky *acceptance criteria* (AC), tedy popisy chování ze zadání, často ve formátu *Given/When/Then*. Testy odvozené z AC tak slouží zároveň jako spustitelná dokumentace požadavků (Solis & Wang, 2011).

Samotný průchod testů ovšem vypovídá primárně o testované implementaci, nikoli o síle testovací sady. První vrstvou hodnocení testů je strukturální coverage (*line, branch*), která ukazuje, jak velkou část kódu testy vykonaly. Coverage sama o sobě ale neříká, zda by testy skutečnou chybu zachytily. Vykonaný řádek bez vhodné aserce projde i tehdy, když implementace dělá něco zcela jiného, než má. Empirická studie na velkém vzorku Java projektů dokládá, že coverage nekoreluje silně s efektivitou testovací sady při detekci chyb (Inozemtseva & Holmes, 2014).

Mutation testing tuto slabinu doplňuje druhým pohledem na sílu testovací sady. Do programu systematicky zavádí drobné syntaktické změny a vytváří tak mutované varianty programu, tzv. mutanty, které simulují běžné programátorské chyby. Poté sleduje, kolik z nich testy odhalí (Papadakis et al., 2019). Vysoký podíl zachycených mutantů indikuje, že testy skutečně ověřují chování, ne jen pokrývají kód.

Udržitelnost a statická analýza

Udržitelnost se na rozdíl od funkční vhodnosti neprojevuje za běhu, ale ve vlastnostech zdrojového kódu, takže její část lze hodnotit i bez spuštění programu.

Cyklomatická složitost kvantifikuje strukturální složitost jedné funkce (McCabe, 1976). Každá rozhodovací konstrukce, například `if`, `while`, `case` nebo logické operátory `&&` a `||`, přidá k hodnotě jedničku, takže funkce bez větvení má složitost 1. Hodnota zároveň udává minimální počet testovacích případů potřebných k pokrytí všech rozhodovacích cest. V praxi se proto často udržuje práh 10 na funkci (McConnell, 2004). Funkce nad tímto prahem statisticky vykazují vyšší míru defektů a obtížnější review.

Vedle metrik složitosti se do statické analýzy řadí i linting, který kontroluje porušení jazykových a projektových konvencí, a typová kontrola, která zachycuje nekonzistence v rozhraních a práci s datovými typy ještě před spuštěním programu. Beller et al. (2016) ukázali na rozsáhlé studii open-source projektů, že tyto nástroje jsou v praxi široce používanou součástí vývojové infrastruktury.

Strukturální nástroje ale pokrývají jen ty aspekty udržitelnosti, které lze formálně vyjádřit pravidlem. Vlastnosti jako čitelnost, srozumitelné pojmenování, oddělení zodpovědností nebo přiměřenost návrhových rozhodnutí přesahují možnosti čistě deterministických nástrojů (McConnell, 2004), protože vyžadují posouzení záměru, kontextu a srozumitelnosti z pohledu jiného vývojáře.

Peer review mimo deterministickou kontrolu

K aspektům, které deterministické nástroje nezachytí, slouží lidská kontrola kódu prováděná peer reviewerem. Jiný vývojář než autor čte navrhovanou změnu, typicky v podobě komentářů u pull requestu, a posuzuje ji před začleněním. Sekce 2.1.1 zmiňuje tutéž aktivitu jako součást procesu změny a traceability. Z produktového hlediska je důležitý obsah review, tedy konkrétní připomínky k pojmenování, dekompozici nebo přiměřenosti návrhových rozhodnutí. Jde právě o vlastnosti, ke kterým se statická analýza nedostane. McIntosh et al. (2016) ukázali, že kód bez review vykazuje měřitelně vyšší míru defektů po vydání. Jak se tato lidská vrstva hodnocení mění s nástupem AI agentů, popisuje sekce 2.2.3.

2.1.3 Zdrojová dimenze

Zdrojovou dimenzí se rozumí vstupy, které jsou nutné k tomu, aby produkt vznikl: čas, lidské úsilí a použití výpočetních prostředků (Fenton & Bieman, 2014). Tato dimenze je samostatná v tom smyslu, že ani z hotového produktu, ani ze záznamu procesu nelze přímo odvodit, kolik úsilí a času vznik stál.

Tradiční nepřímé ukazatele a jejich limity

V tradičním softwarovém inženýrství se zdroje často měří přes nepřímé kvantitativní ukazatele. Modely odhadu pracnosti jako COCOMO (Boehm, 1981) a jeho aktualizace COCOMO II (Boehm et al., 2000) odhadují pracnost na základě velikosti projektu a sady korekčních faktorů, v agilních týmech se používají relativní odhady (*story points*) a ukazatele průchodu prací (*cycle time*, *lead time*). Predikční přesnost těchto modelů je ale dlouhodobě sporná. Systematický přehled 304 studií ukazuje, že odhady pracnosti běžně vykazují vysokou chybovost a že strojově spočítané modely často neporazí lidský odhad zkušeného inženýra (Jørgensen & Shepperd, 2007). Společným omezením všech těchto měřítek je navíc to, že redukuje vícerozměrný jev (čas, úsilí, kvalita výstupu, dopad na tým) na jedinou osu, a samy o sobě nezachytí, zda je vývoj efektivní v širším slova smyslu.

Víceosé měření a přechod ke zdrojům agentů

Reakcí na tato omezení jsou rámce DORA (Forsgren et al., 2018) a navazující SPACE (Forsgren et al., 2021), které explicitně varují před redukcí výkonnosti na jedinou metriku a místo toho pracují s několika osami současně. Stejný princip víceosého měření lze přenést i na hodnocení agenta, obsah zdrojové dimenze se však mění. U autonomního agenta ustupuje lidské úsilí v klasickém Fentonově smyslu do pozadí. Relevantními ukazateli se stávají spotřebované tokeny, reálný čas běhu a stabilita session, k níž v agentním prostředí přibývá kontextové okno jako vzácný zdroj. U agentů se tak zdrojová dimenze přesouvá od lidského úsilí k měřitelným stopám běhu a využití kontextu.

2.2 AI coding agenti

U AI coding agentů zůstávají relevantní stejné tři dimenze měření jako v tradičním vývoji, mění se však mechanismus jejich vzniku. Kód nevzniká přímo prací člověka, ale interakcí jazykového modelu, instrukcí a nástrojů. Tomu odpovídají i produktové vzorce kódu, procesní stopa v repozitáři a nepřímé ukazatele spotřeby zdrojů.

2.2.1 Základní pojmy

Velký jazykový model (*large language model*, LLM) je neuronová síť trénovaná na rozsáhlých textových korpusech (Vaswani et al., 2017). Na základě předchozího kontextu predikuje pravděpodobný následující token, a tím generuje koherentní text, včetně zdrojového kódu. Samotný LLM je pasivní. Odpovídá na dotazy, ale nemůže samostatně číst soubory, spouštět příkazy ani měnit stav projektu. Predikce navíc není deterministická, takže ani explicitní instrukce nezaručuje, že ji model dodrží.

LLM-based agent model rozšiřuje o schopnost interagovat s prostředím (Liu et al., 2024). Plánuje kroky, volá nástroje prostřednictvím tzv. *tool calls* (Schick et al., 2023) a na základě pozorovaného výsledku rozhoduje o dalším kroku v cyklu Thought → Action → Observation (Yao et al., 2023). Vrstvu, která *tool calls* zprostředkovává a spravuje stav session, tvoří *harness*, orchestrační vrstva tvořící rozhraní mezi modelem a prostředím. Harness agentovi zpravidla nabízí minimálně čtení souboru, zápis souboru a spuštění shell příkazu. Přes tyto nástroje může v repozitáři spouštět testy, používat verzovací systém i volat externí API. Tyto akce po sobě zanechávají měřitelné stopy (commity, větve, pull requesty) a současně otevírají prostor pro typická selhání, například nesprávně zvolenou větev, neoprávněnou modifikaci existujícího testu nebo vynechaný krok pracovního postupu.

Harness výrazně ovlivňuje výkon agenta. Yang et al. (2024) v open-source nástroji SWE-agent ukazují, že i drobné úpravy rozhraní mezi agentem a prostředím (*agent-computer interface*) posouvají úspěšnost na reálných úlohách o jednotky procentních bodů. Harness také zaznamenává průběh session jako strukturovaný transcript, který slouží jako audit log běhu agenta. Spolu s obsahem kontextového okna tvoří dvě odlišné vrstvy řízení chování. Harness působí skrze design rozhraní a *tool calls*, instrukce skrze textový obsah, který agent dostane jako vstup.

Předmětem této práce jsou *autonomní coding agenti*, tedy systémy, které na základě specifikace a instrukcí samostatně implementují softwarový projekt napříč soubory, příkazy a verzovací historii (Guo et al., 2025). Na rozdíl od autocomplete nástrojů nebo code-review botů v CI nevytvářejí pouze dílčí návrhy či hodnocení, ale provádějí vícekový vývojový postup.

2.2.2 Jak agenti mění softwarové inženýrství

Přechod od lidského vývojáře k autonomnímu agentovi posouvá softwarové inženýrství ve všech třech dimenzích zavedených v sekci 2.1. Jin et al. (2024) v přehledu 124 studií ukazují, že agenti pronikají do šesti oblastí SE, konkrétně požadavků, generování kódu, autonomního rozhodování, návrhu, generování testů a údržby. Pro generování kódu a testů jsou tato omezení zvláště viditelná. Agent pracuje s omezeným kontextovým oknem, nemá implicitní znalost projektu a jeho výstup zůstává pravděpodobnostní. Selhání se proto v každé dimenzi projevují trochu jinak.

Procesní stránka se mění v tom, kdo provádí jednotlivé kroky, a v tom, jakou stopu po sobě zanechávají. Typická procesní selhání dokládají pozorování agent-generovaných pull requestů. Ty se od těch lidských liší mírou nadbytečnosti, konzistencí commit zpráv a ochotou reviewera je přijmout (Ehsani et al., 2026; Watanabe et al., 2025).

Produktová stránka zůstává vymezena stejnými charakteristikami (viz sekce 2.1.2), ale typické vzorce agent-generovaného kódu jsou jiné než u lidského vývojáře. B. Li et al. (2026) na 1210 začleněných agentem vytvořených PR z reálných repozitářů zjistili, že i přijaté patche nesou vyšší výskyt code smells, tedy signálů špatné vnitřní struktury kódu, duplikací a strukturálních problémů, které samotné začlenění změny neodhalí. Tato produktová selhání se

proto v literatuře hodnotí kombinací automatizovaných benchmarků (sekce 2.2.3) s post-hoc statickou analýzou a s hodnocením od LLM-judgů či lidských reviewerů.

Zdrojová stránka se u agentů opírá o jiné nepřímé ukazatele. Na místě člověko-hodin stojí spotřebované tokeny a čas běhu na úlohu, kontextové okno a rate limit vystupují jako vzácné zdroje, které ovlivňují, kolik úloh a jak dlouhých agent zvládne. Zdrojová selhání pak souvisejí s neefektivním řízením kontextu. Lindenbauer et al. (2025) pozorují, že jednoduché skrývání starších výstupů z tool calls (*observation masking*) dosáhne srovnatelné úspěšnosti jako jejich shrnutí pomocí LLM za polovinu nákladů.

Jak se schopnosti agentů konkrétně měří dnes a jaké mezery v tomto měření zůstávají, shrnuje následující sekce.

2.2.3 Hodnocení schopností agentů

Hodnocení autonomních coding agentů prošlo několika vlnami. Foundational benchmark SWE-bench (Jimenez et al., 2024) testuje schopnost agenta vytvořit patch na 2 294 úlohách z reálných GitHub repozitářů, kde agent dostane popis problému (GitHub issue) a má vytvořit patch, který projde testy. Aktuálně se jako standard prosazuje Terminal-bench (Merrill et al., 2026), 89 tvrdých úloh v reálných terminálových prostředích, kde i frontier modely zatím překračují 50 % úspěšnosti jen výjimečně. Starší HumanEval (Chen et al., 2021) zůstává referencí pro generování funkčně správného kódu na izolovaných programovacích úlohách. Společně tyto benchmarky měří především funkční úspěšnost, tedy zda patch projde nebo neprojde. Takové měření je užitečné pro srovnání modelů, ale samo nezachycuje review přijatelnost, udržitelnost ani auditovatelnost postupu, na které upozorňují studie shrnuté v kapitole 1.

Novější benchmarky tuto perspektivu rozšiřují. SWE-bench Pro (Scale AI, 2025) přidává strukturované požadavky a FeatureBench (Zhou et al., 2026) hodnotí end-to-end implementaci celých funkcí. Evaluace sedmi agentních frameworků jde ještě dál a hodnotí je na třech osách (úspěšnost, efektivita reasoning kroků a tokenová režie) (Z. Yin et al., 2025). Její pohled na proces je ale omezen na trasu reasoning kroků agenta a strukturální kvalita kódu v hodnocení chybí. Žádný z uvedených benchmarků tedy systematicky neměří kvalitu procesu (jak agent organizoval práci), kvalitu produktu nad rámec funkční korektnosti (udržitelnost, čistota) a spotřebu zdrojů současně. Tato mezera je důležitá pro tuto práci, protože vyžaduje sadu metrik, která pokrývá proces, kvalitu produktu a spotřebu zdrojů současně.

LLM-as-judge

Některé dimenze kvality, jako srozumitelnost commit zpráv, kvalita dokumentace nebo vhodnost návrhových vzorů, nelze spolehlivě hodnotit deterministickými nástroji. Jedním z přístupů je využití LLM jako hodnotitele (*LLM-as-judge*). Zheng et al. (2023) ukázali, že LLM dokáže

hodnotit kvalitu výstupů na úrovni srovnatelné s lidskými hodnotiteli, ale identifikovali tři systematické biasy: *position bias* (preferuje odpověď na určité pozici), *verbosity bias* (preferuje delší odpovědi) a *self-enhancement bias* (preferuje vlastní výstupy). Panickssery et al. (2024) prokázali korelaci mezi schopností modelu rozpoznat vlastní výstupy a silou preference těchto výstupů, což naznačuje kauzální vztah mezi self-recognition a self-preference. Verga et al. (2024) navrhli mitigaci formou panelu diverzních modelů (PoLL), který dosahuje lepší shody s lidským hodnocením než jednotlivý model. Praktickou implikací je volba hodnotícího modelu z odlišné modelové rodiny, než ve které pracuje hodnocený agent.

Test oracle problem

Test oracle problem je klasická obtíž testování, tedy kdo nebo co určuje správný výsledek, proti kterému se test porovnává? U autonomního agenta se zostřuje tím, že agent generuje implementaci i testy současně. Pokud testy odvodí z toho, jak se kód právě chová, jejich průchod potvrzuje jen vzájemnou konzistenci, ne shodu se specifikací. Mathews and Nagappan (2024) dokládají, že generátory testů zahazují právě ty testy, které by chybu odhalily. Až 68,1 % výsledných testovacích sad pak validuje chybné chování místo jeho odhalení. Obranou je odvozovat očekávané hodnoty ze specifikace (*spec-first TDD*, sekce 2.1.1). Specifikace tím vystupuje jako externí oracle, tedy zdroj očekávaných výsledků, který agent nemůže přepsat.

Benchmarky i doplňkové mechanismy z této sekce hodnotí převážně izolované úlohy a jednotlivé aspekty výstupu. Pro vyhodnocení agenta v kontextu jedné případové studie je třeba metrik pokrývajících dimenze produktu, procesu i zdrojů současně.

2.2.4 Instrukce jako nezávislá proměnná

Disciplína zabývající se formulací textových vstupů pro velký jazykový model se v literatuře označuje jako *prompt engineering* a její rozšíření o správu celého kontextového okna jako *context engineering*. Tato práce používá nadřazený pojem *instrukce*, který obě tyto disciplíny zastřešuje.

Autonomní agent nemá *tacit knowledge*, tedy implicitní znalosti, které lidský vývojář získává zkušeností, například jaké konvence tým dodržuje, proč je kód strukturovaný určitým způsobem nebo jaké chyby se v projektu opakují. Nonaka and Takeuchi (1995) popisují převod tacit knowledge na explicitní jako klíčový proces v organizacích. Pro agenty je tento převod nutný v plném rozsahu. Veškerý kontext, který agent potřebuje, musí být zapsán explicitně. Hassan et al. (2025) tuto dualitu formalizují v rámci frameworku SASE, který rozlišuje dva režimy vývoje. V *SE4H* (*Software Engineering for Humans*) je vývojářem člověk a kontext čerpá z meetingů, code review a zkušenosti. V *SE4A* (*Software Engineering for Agents*) je vývojářem autonomní systém, který pracuje výhradně s tím, co dostane jako vstup.

Scaffolding a soubory s instrukcemi

Scaffolding označuje strukturované artefakty v prostředí agenta, které jeho chování řídí. V praxi jde nejčastěji o soubory s instrukcemi umístěné v repozitáři (např. `AGENTS.md`, `CLAUDE.md`), které se v poslední době prosazují jako de facto standard napříč většinou coding agentů. Tyto soubory definují roli agenta, pracovní postup, konvence a omezení. Na rozdíl od jednorázových promptů jsou tyto soubory verzované a iterativně upravované, což z nich činí „living documents“, jejichž vývoj lze zpětně sledovat.

Komponenty instrukcí

Mao et al. (2025) analyzovali 2 163 produkčních prompt šablon a identifikovali sedm typů komponent: Role, Directive, Context, Workflow, Output, Constraints a Examples. Zjistili, že Role a Directive se nejčastěji objevují na začátku dokumentu a že Workflow (procedurální kroky) je nejúčinnější komponentou pro složité úkoly. Toto zjištění potvrzuje X. Li et al. (2026), kteří ukazují, že procedurální instrukce jsou efektivnější než popisná dokumentace.

Empirie účinnosti

Dostupná evidence o účinnosti instrukčních souborů je smíšená a závisí na tom, co se měří a jaký je obsah souboru. V efektivitě běhu Lulla et al. (2026) pozorují, že přítomnost `AGENTS.md` zkracuje mediánovou dobu, kterou agent potřebuje na splnění úlohy, o 28,6 %. V úspěšnosti řešení úlohy ale rozhoduje původ obsahu. LLM-generované instrukce úspěšnost snižují a vývojářem psané přinášejí jen marginální zlepšení (Gloaguen et al., 2026), kdežto kurátorovaný obsah v benchmarku SkillsBench zvyšuje úspěšnost o 16,2 procentních bodů a automaticky generovaný nemá žádný efekt (X. Li et al., 2026). Rozhodující je tedy obsah, struktura a relevance, ne pouhá přítomnost souboru. Účinnost jednotlivých typů obsahu přitom dosud nebyla izolována.

Specifita a citlivost

Instrukce musí být dostatečně konkrétní, ale ne nutně co nejdelší. Kim (2025) v benchmarku DETAIL a Zi et al. (2025) na úlohách s kódem shodně ukazují, že vyšší *specifita* zvyšuje úspěšnost agenta, přílišný detail však může omezit jeho reasoning. Instrukce se tak pohybují po ose od volné nápovědy po explicitní příkaz, přičemž optimální bod závisí na úloze i modelu.

Modely jsou kromě toho citlivé na samotnou formulaci. Razavi and Fard (2025) dokládají, že drobné změny promptu mohou dramaticky změnit chování modelu. Breunig (2025) na to navazuje praktickým doporučením. Když agent pravidlo ignoruje, nepomůže ho zopakovat, je třeba ho přeformulovat tak, aby se stalo součástí pracovního postupu.

Mechanismy účinku

Výzkum promptingu navíc naznačuje, že instrukce nepůsobí na chování modelu jedním mechanismem. Wei et al. (2022) ukazují, že i krátká nápověda (“think step by step”) aktivuje reasoning, který se bez ní neprojeví, a Min et al. (2022) dokládají, že ukázkové příklady vložené do kontextu modelu působí spíš jako strukturální vodítka než jako přímé příkazy. Instrukce tedy nemusí působit jen jako explicitní příkazy. Mohou také určit pořadí kroků, formát výstupu nebo vyvolat reasoning, který se bez vhodného podnětu neprojeví. Která forma instrukce bude v konkrétním případě účinnější, nelze rozhodnout předem.

Otevřenou otázkou proto není jen to, zda instrukce obecně pomáhají, ale jak jejich obsah systematicky navrhovat a jak poznat, které složky skutečně mění chování agenta. Z teoretické části proto plynou dvě východiska: současné hodnocení agentů nepokrývá proces, kvalitu produktu a spotřebu zdrojů společně a návrh instrukcí nemá ustálený systematický postup.

3. Metodika

3.1 Výzkumný přístup

3.1.1 Volba výzkumné strategie

Z tří cílů formulovaných v kapitole 1 plynou společné nároky na metodiku. Cíl 1, návrh sady metrik pokrývající proces, kvalitu produktu a zdroje, vyžaduje sadu metrik citlivou na změny v chování. To předpokládá porovnání více běhů s různými variantami instrukcí. Cíl 2, iterativní postup návrhu instrukcí řízený těmito metrikami, vyžaduje navíc, aby běhy tvořily sekvenci, v níž lze sledovat vztah měření $\rightarrow$ diagnóza $\rightarrow$ úprava $\rightarrow$ další měření. Cíl 3, ablace složek instrukcí, stojí na schopnosti porovnat běhy lišící se v jediné složce. Všechny tři cíle tedy společně kladou na metodiku dva požadavky. Prvním je opakované spouštění agenta s různými variantami instrukcí na témže projektu, druhým podrobná analýza každého běhu napříč třemi dimenzemi (proces, produkt, zdroje).

Tomuto profilu odpovídá případová studie, která v empirickém software engineering výzkumu slouží k detailnímu zkoumání jevu v jeho reálném kontextu (Runeson & Höst, 2009). Řízený experiment s počtem běhů potřebným pro statistickou izolaci by byl vzhledem k ceně jedné iterace (tisíce tokenů, desítky minut) nepraktický. Přednost proto dostává hloubka před šířkou, tedy jeden projekt s detailní analýzou každého běhu (git historie, transcript, naměřené metriky). R. K. Yin (2018) pro tento typ výzkumu používá pojem *analytická generalizace*. Na jednom případě ukazujeme princip, který lze dále ověřovat na dalších případech. Praktický přínos této práce spočívá v ověření, že navrženou sadu metrik a iterativní postup lze aplikovat na jednom případě, zatímco analytická generalizace vymezuje způsob, jak opatrně číst přenositelné principy. Všechny běhy vycházejí ze stejné specifikace projektu, liší se verzí instrukcí, výslednou git historií a naměřenými hodnotami metrik.

Touto volbou se práce vědomě staví mimo benchmark paradigma (Jimenez et al., 2024), které srovnává mnoho modelů na mnoha úlohách s agregovanými skóre. Cílem této studie není sestavit žebříček modelů, ale podrobné porozumění tomu, jak iterativní úpravy instrukcí mění chování jednoho agenta na jednom projektu. To je úloha pro případovou studii, ne pro benchmark.

Pilotní fáze

Případová studie postupuje ve dvou fázích. Pilotní fáze slouží k nalezení použitelné sady instrukcí. První běh používá výchozí `AGENTS.md`, tedy sadu instrukcí odvozenou z literatury o instrukcích pro agentní vývoj. Další běhy už vznikají iterativně. V každém běhu se sleduje, zda agent dodržel požadovaný postup a jak kvalitní řešení vytvořil. Naměřené odchylky se

potom používají jako podklad pro další úpravu instrukcí. Pilotní fáze tak ukazuje, zda metriky dávají dostatečně konkrétní zpětnou vazbu pro řízené zlepšování instrukcí.

Ablace

Ablační fáze vychází z použitelné sady instrukcí nalezené v pilotu. Z této sady se odebere vybraná složka a sleduje se, zda se změní chování agenta. Ablace tedy netvoří další ladicí cyklus, ale kontrolované porovnání plné verze instrukcí s její zjednodušenou variantou. Každá ablace se provádí ve dvou nezávislých bězích se stejným nastavením, aby bylo možné odlišit opakující se efekt změny od přirozené variability nedeterministického modelu. Ani dva běhy neposkytují statistickou průkaznost, a výsledky proto mají indikativní, ne kauzální povahu.

3.1.2 Příklad a jednotky analýzy

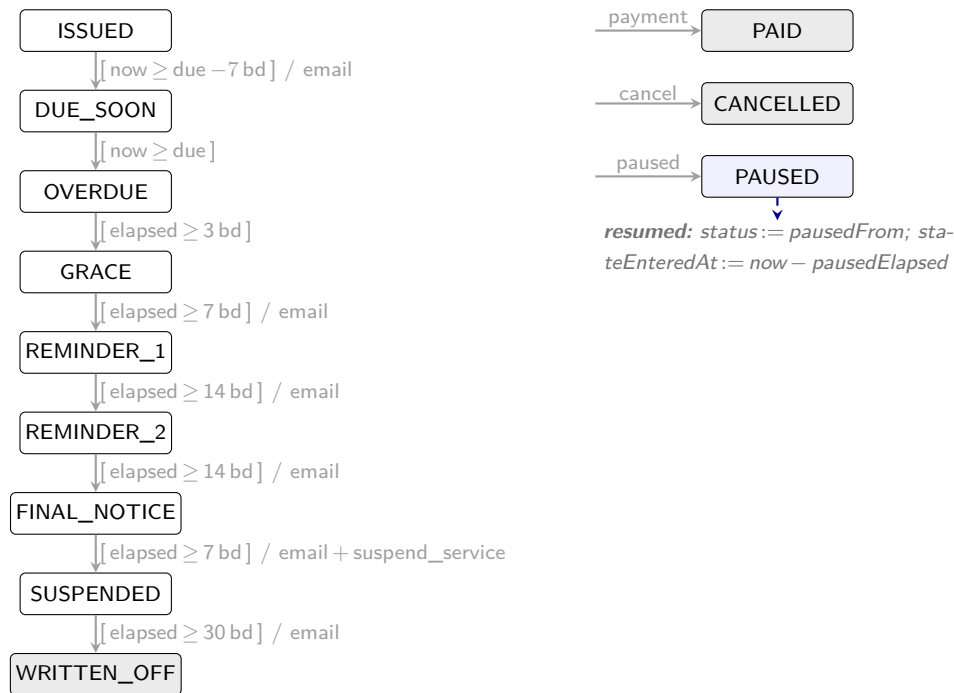
Případem v Yinově smyslu (R. K. Yin, 2018) je projekt systému upomínek faktur. *Jednotkami analýzy* jsou jednotlivé běhy agenta nad tímto projektem (běhy z pilotní fáze a z ablačních variant), z nichž každý poskytuje vlastní data pro hodnocení napříč třemi dimenzemi (proces, produkt, zdroje). Případová studie potřebuje projekt, na kterém lze opakovaně spouštět agenta s různými instrukcemi a objektivně měřit výsledky. Projekt musí mít deterministickou logiku (stejný vstup vždy dá stejný výstup), aby bylo možné ověřit korektnost automatizovanými testy a mutation testingem. Zároveň musí být dostatečně malý pro více experimentálních běhů, ale obsahovat reálné nuance (hraniční případy, doménová pravidla), aby měl agent co řešit.

Zvolený případ představuje aplikaci pro automatické odesílání připomínek k nezaplaceným fakturám. Obsahuje stavový automat (nová, po splatnosti, upomínaná, eskalovaná), časové výpočty (pracovní dny, ochranné lhůty) a pravidla pro eskalaci. Současně jde o úlohu dostatečně bohatou na netriviální rozhodování, ale stále zvládnutelnou pro opakované běhy. Agent musí kombinovat stavové přechody, časové výpočty a testování hraničních případů, takže metriky zachycují víc než průchod jednoduchou CRUD implementací.

Specifikace projektu tvoří fixní zadání, které agent dostává v Issue #1. Issue má dvě části: **Requirements** (co business potřebuje) a **API Contract** (co musí implementace exportovat). API kontrakt je současně jediným technickým omezením implementace a vstupem pro metriku Q1 (API contract match). Plné znění specifikace je součástí doplňujících materiálů odkazovaných v příloze B.

Doménová logika

Systém eskaluje neuhrazené faktury sérií stavů od přátelské upomínky před splatností přes stupňující se urgentnost až po pozastavení služby a odpis pohledávky. Každá faktura má vlastní



Obrázek 1: Stavový automat systému upomínek faktur.

nezávislou instancí, časové prodlevy se počítají v pracovních dnech (víkendy a konfigurovatelné svátky se přeskakují automaticky v aritmetice timeoutů, nejsou samostatným stavem). Hlavní eskalační větev má 9 stavů spojených 8 časovými přechody (obrázek 1). Popisky šipek používají notaci `[guard] / effect`. Guard v hranatých závorkách udává timeout v pracovních dnech (*bd* = business days) od vstupu do předchozího stavu, effect za lomítkem je akce vrácená volajícímu (typicky odeslání e-mailu, u přechodu do **SUSPENDED** navíc pozastavení služby). Eskalační přechody spouští událost `tick` při splnění guardu. Alternativně `manual_advance` posune stav na další bezpodmínečně (bez čekání na timeout, v diagramu nezakreslen). Vedle hlavní větve jsou ze všech aktivních stavů kdykoliv dostupné **PAID** a **CANCELLED** (z 9 stavů, vše kromě terminálů) a **PAUSED** (jen ze 6 aktivních: **OVERDUE** až **SUSPENDED**).

API kontrakt

API kontrakt určuje, co musí agentova implementace poskytovat, aby ji bylo možné porovnávat s referenční sadou behavioral testů (Q2). Systém je čistá funkce `process(state, event, now)`, která pro daný stav, událost a aktuální čas vrátí nový stav a seznam akcí. Pevné jsou názvy 12 stavů (9 v hlavní eskalační větvi a 3 boční: **PAID**, **CANCELLED**, **PAUSED**), 6 typů událostí (`tick`, `payment_received`, `invoice_cancelled`, `dunning_paused`, `dunning_resumed`, `manual_advance`) a 3 typy akčních deskriptorů (`send_email`, `suspend_service`, `resume_service`). Stav nese kromě `status` pole pro datum splatnosti, čas vstupu do aktuálního stavu, konfiguraci a při pauze `pausedFrom` s `pausedElapsed`. Vnitřní implementace (modulární rozdělení, funkční vs. objektový styl) zůstává na agentovi. Kontrakt je úzký právě v tom, co testy potřebují volat a pozorovat,

a všechno ostatní nechává otevřené.

Implicitní pravidla domény

Stavový automat je sám o sobě lineární a implementačně přímočarý. Obtížnost úlohy leží ve dvou implicitních pravidlech, která specifikace klade na chování implementace. Jejich přehlédnutí vede ke kódu, který může projít triviálními případy, ale selhává na hraničních situacích. Za prvé, pause/resume musí přes pole `pausedElapsed` zachovat uplynulý čas. Bez něj by se obnovení běhu muselo řešit posunem `stateEnteredAt` zpět v čase. Takové řešení funguje pro jednu pauzu, ale selhává při opakovaném pause/resume, protože druhá pauza ztratí historii první. Za druhé, všechny timeouty se počítají v pracovních dnech, takže **+14 bd** od pátku není čtrnáct kalendářních dnů a hranice mezi víkendem a pondělím jsou off-by-one zóna. Obě pravidla jsou plně určená API kontraktem. Jejich zachycení ale vyžaduje doslovné čtení typů a doménového slovníku, ne pouze odvozování z diagramu automatu.

Acceptance criteria

Specifikace obsahuje 25 acceptance criteria ve formátu Given/When/Then. Pokrývají sedm okruhů: časové přechody eskalace, platby, terminální stavy, storno, pauza/obnovení, manuální postup a výpočet pracovních dní. Ukázka jednoho kritéria z okruhu pauza/obnovení:

```
Given an invoice is in PAUSED state
When a dunning_resumed event occurs
Then the state transitions back to the state it was in before pausing
And the timeout resumes from where it left off
```

Formát Given/When/Then usnadňuje převod požadavků do referenčních testů (Q2 (úspěšnost referenčních testů)) i mapování acceptance criteria na agentovy vlastní testy (Q4 (AC coverage)). Specifikace obsahuje anglický doménový slovník s 9 pojmy, například upomínkový proces (*dunning*), ochrannou lhůtu (*grace period*), pracovní dny (*business days*) a popis akce (*action descriptor*). Tyto pojmy sjednocují terminologii mezi specifikací a implementací.

Out of scope

Specifikace explicitně vylučuje šest oblastí: opakování plateb, úroky a poplatky z prodlení, částečné platby, odesílání e-mailů, plánování (cron) a persistenci dat. Sekce out of scope omezuje riziko, že agent rozšíří řešení nad rámec sledovaného zadání, a drží úlohu na úrovni čisté business logiky.

Referenční testy jako oracle

Pro metriku Q2 (úspěšnost referenčních testů) (sec 3.2.2) byla souběžně se specifikací připravena sada 42 referenčních testů metodou spec-first TDD (Mathews & Nagappan, 2024): očekávané hodnoty pochází ze specifikace, ne z pozorování kódu. Testy jsou agentovi neviditelné a spouští se až po běhu samostatným skriptem, takže slouží jako externí oracle, tedy zdroj očekávaných výsledků, který agent nemůže přepsat (sec 2.2.3). Mutation testing nad referenční testovací sadou dosáhl 91,9 %, což indikuje, že referenční testy samy detekují cílené poruchy v doménové logice. Tato hodnota slouží jako pojistka kvality měřicího nástroje, ne jako srovnávací základna pro agentovy běhy.

V tomto designu vystupují tři roviny, které se snadno zaměňují. *Instrukce* v souboru `AGENTS.md` jsou nezávislou proměnnou, která se mezi běhy mění a jejíž dopad se sleduje. *Chování agenta* (jak implementuje, jak strukturuje práci a jaký kód vytváří) je závislou proměnnou, kterou měříme. Systém upomínek faktur slouží jako testovací prostředí, na kterém chování pozorujeme. Není předmětem zkoumání.

3.1.3 Fixní proměnné

Aby bylo možné měřit vliv instrukcí, musí být všechno ostatní co nejstabilnější. Hlavní měněnou proměnnou mezi běhy je obsah souboru `AGENTS.md`, tedy procedurálních instrukcí, které definují pracovní postup, omezení a quality gates. Tento design negarantuje dokonalou izolaci všech vlivů. Jeho cílem je omezit zjevné vedlejší vstupy a udělat jejich případný vliv auditovatelný. Na rozdíl od generických context files, které typicky popisují adresářovou strukturu a build příkazy (Gloaguen et al., 2026), jde o fokusované instrukce s konkrétními procedurami (X. Li et al., 2026).

Fixní proměnné jsou:

Prázdné GitHub repo obsahuje pouze `AGENTS.md` a konfiguraci agenta. Žádný existující kód, testy ani `package.json`. Agent musí inicializovat projekt a zvolit strukturu sám.

Specifikace projektu (popsaná v sekci 3.1.2) zůstává napříč všemi běhy beze změny a je dodávána agentovi v GitHub Issue #1 jako hlavní věcný vstup vedle instrukcí.

Modely jsou mezi běhy fixní. Agentní běhy provádí MiniMax-M2.5, zvolený kvůli nízké ceně a rychlosti při opakovaných agentních bězích. Cílem práce je vyhodnotit sadu metrik a iterativní postup, ne srovnávat výkon modelů. Konkrétní agentní model proto slouží jako součást experimentálního nastavení. Pro judge-based metriky je fixně použit GLM-5 (Zhipu AI), záměrně z jiné modelové rodiny než agent. Volba vychází z nízké ceny a biasů LLM-as-judge popsanych v sekci 2.2.3. Teplota modelu nebyla mezi běhy explicitně nastavena. Agent běžel s výchozí konfigurací poskytovatele.

Běhové prostředí (Docker container) zajišťuje, že agent nemá přístup ke globální kon-

figuraci nástroje, MCP serverům ani rozšířením nainstalovaným na hostitelském systému. Izolace tím významně omezuje vliv vedlejších vstupů z prostředí, ale nevylučuje všechny zdroje variability. Sdílený autentizační svazek zajišťuje konzistentní přístup k API napříč běhy.

Harness OpenCode je CLI nástroj, který orchestruje běh agenta. Spravuje session, zprostředkovává volání modelu, zpřístupňuje nástroje (čtení a zápis souborů, shell, vyhledávání) a zaznamenává transcript. Harness sám není měřeným chováním. Měří se to, co agent přes něj udělá. Verze nástroje je pinovaná v Docker image, konfigurace agenta (povolené nástroje, MCP servery) je součástí auditovatelného balíku v repozitáři práce. Volba OpenCode je dána open-source licencí (umožňuje pinování verze a záznam interní telemetrie), strojově čitelným exportem session a přístupem do interní databáze, ze které čerpá metrika E3 (počet kompakcí kontextu). Vlastnosti harness, které ovlivňují měření, jsou diskutovány u jednotlivých metrik, zejména tichý retry po selhání API (sekce 5.3) a automatická kompakce kontextu (metrika E3).

System prompt agenta (`build.md`) nahrazuje výchozí system prompt nástroje OpenCode, jehož instrukce o verzování konfliktovaly s procesními požadavky případové studie. Vlastní system prompt obsahuje pouze obecné kódové konvence (styl, bezpečnost, zákaz komentářů) a odkaz na `AGENTS.md`. Veškeré procesní instrukce jsou výhradně v `AGENTS.md`, aby bylo možné změny procesního chování přisuzovat této vrstvě obhajitelněji než při jejich rozptýlení mezi více vstupů.

3.2 Sada metrik

Každý experimentální běh probíhá tak, že agent dostane prázdné GitHub repo, specifikaci v Issue #1 a instrukce v `AGENTS.md`, a autonomně implementuje zadaný projekt. Výstup jednoho běhu (kód, testy, git historie) měříme sadou 19 metrik. Proceduru jednoho běhu (záznam, vyhodnocení a diagnostickou úpravu instrukcí) popisuje sekce 3.3.

Sada metrik pokrývá tři otázky, tedy jak agent pracoval, co vyrobil a za jakou cenu. Tyto tři otázky odpovídají taxonomii proces / produkt / zdroje popsané v sekci 2.1. Naše sada obsahuje 19 metrik ve třech kategoriích:

- P – proces (P1–P8)** Dodržel agent předepsaný postup? Měříme dodržování pravidel pracovního postupu z instrukcí (P1–P5) a kvalitu procesních výstupů, jako jsou zprávy commitů, popisy issue a popisy PR (P6–P8).
- Q – kvalita produktu (Q1–Q8)** Je kód kvalitní? Měříme funkční korektnost (Q1–Q2), kvalitu testů (Q3–Q4) a udržitelnost kódu (Q5–Q8).
- E – zdroje (E1–E3)** Za jakou cenu? Zaznamenáváme spotřebované tokeny (E1), čas (E2) a počet kompakcí kontextu (E3).

Tři osy klasifikace metriky

Každou metriku popisujeme třemi vlastnostmi. První je kategorie procesní metriky (P1–P8), produktové metriky (Q1–Q8) nebo metriky zdrojů (E1–E3), tedy zda metrika sleduje proces, produkt nebo zdroje. Druhá je způsob měření. Deterministické metriky počítá skript, parser logu nebo statická analýza, zatímco judge-based metriky hodnotí LLM-as-judge podle hodnoticího schématu. Třetí vlastností je role v hodnocení. Procesní a produktové metriky mají exit kritéria a slouží k posouzení, zda byl běh úspěšný. Metriky zdrojů E1–E3 exit kritéria nemají. Jejich hodnoty se pouze zaznamenávají a slouží jako kontext pro porovnání spotřeby zdrojů a stability kontextu během běhu.

LLM-as-judge protokol

Pět metrik (P6, P7, P8, Q4, Q8) hodnotí vlastnosti, které nelze spolehlivě extrahovat deterministickým skriptem. Proto používají LLM-as-judge s modelem GLM-5 (Zhipu AI). GLM-5 je z jiné modelové rodiny než agentní MiniMax-M2.5, protože modely mají tendenci zvýhodňovat výstupy svého vlastního druhu, tedy vykazují self-preference bias (Panickssery et al., 2024). Volba hodnotitele z jiné rodiny tento bias snižuje (Verga et al., 2024).

Judge dostává výstupy běhu, například zdrojový kód, zprávy commitů, popisy issue a popisy PR. Hodnotí je podle pevného hodnoticího schématu (*scoring rubric*), které určuje posuzované dimenze a význam stupňů 1 až 3 (1 = nejnižší kvalita, 3 = nejvyšší kvalita). Stejný prompt, hodnoticí schémata i model GLM-5 se používají ve všech bězích, aby bylo hodnocení mezi iteracemi srovnatelné.

Judge-based metriky mají v této práci podpůrnou roli. Umožňují strukturovaně zachytit vlastnosti, které deterministické nástroje nepokrývají, ale nejsou interpretovány jako plně objektivní měření.

3.2.1 Procesní metriky (P1–P8)

Procesní metriky měří *jak* agent pracuje. Instrukce v `AGENTS.md` definují strukturovaný vývojový postup. Agent má odvozovat práci ze specifikace, organizovat ji přes issues a větve, psát testy před implementací a dokumentovat rozhodnutí ve zprávách commitů a popisech PR.

Tato skupina metrik převádí procesní dimenzi z teoretických východisek do stop, které po sobě agent zanechává v repozitáři a na GitHubu. Proto sleduje pořadí práce, vazbu mezi issues, větvemi a pull requesty, test-first postup a kvalitu procesních záznamů.

P1–P5: dodržení procesních pravidel (binární)

Tabulka 1 shrnuje deterministické procesní metriky. Všechny mají binární výsledek splněno/-nesplněno.

Tabulka 1: Procesní metriky dodržení pravidel P1–P5

Kód	Co měří	Jak se měří	Exit kritérium
P1	Issues vznikla před kódem	Porovnání <code>created_at</code> nejstaršího issue vytvořeného agentem (GitHub API) s časem prvního commitu obsahujícího soubory v <code>src/</code> nebo <code>tests/</code> .	kontrola projde
P2	Branch per issue	Počet vzdálených větví (bez <code>main</code>) vůči počtu issues, podmínka $branches \geq issues$ (žádné slučování více issues do jedné větve).	kontrola projde
P3	Test-first	Primárně se kontroluje, zda existuje commit s prefixem <code>test:</code> před commitem <code>feat:</code> . Jako doplňková kontrola se z git historie ověřuje, zda první změna v <code>src/</code> nepředchází první změně v <code>tests/</code> .	kontrola projde
P4	PR linkován na issue	Tělo každého PR (GitHub API) obsahuje regex <code>Closes #N</code> .	kontrola projde
P5	Žádné modifikace existujících testů	Metrika sleduje, zda agent měnil již existující testy v adresáři <code>tests/</code> . Kontrola používá <code>git diff --diff-filter=M</code> . Nově přidané testy se nezapočítávají, protože rozšiřování testovací sady je v zadání povoleno.	kontrola projde

P6–P8: kvalita procesních výstupů (LLM-as-judge)

Tabulka 2 shrnuje judge-based procesní metriky.

Tabulka 2: Judge-based procesní metriky P6–P8

Kód	Co hodnotí	Škála	Exit kritérium
P6	Kvalita zpráv commitů	1–3 (jeden logický záměr na commit, konvenční prefix, srozumitelnost co a proč bylo změněno).	$\geq 2/3$
P7	Kvalita popisů issue	1–3 (jasnost scope, přítomnost acceptance criteria, dostatečnost pro implementaci).	$\geq 2/3$
P8	Kvalita popisů PR	1–3 (odkaz na issue, popis co a proč, dostatečnost pro code review).	$\geq 2/3$

3.2.2 Produktové metriky (Q1–Q8)

Produktové metriky měří *co* agent vyrobil. Z osmi charakteristik ISO/IEC 25010 (sekce 2.1.2) v této sadě sledujeme dvě: *funkční vhodnost* (Q1–Q4: agent vytváří správné řešení) a *udržovatelnost* (Q5–Q8: řešení je snadno udržovatelné).

nost (Q5–Q8: agent vytváří kód vhodný k dalšímu pochopení a rozvoji). Zbylé charakteristiky ISO/IEC 25010 by vyžadovaly jiný typ úlohy. Bezpečnost předpokládá autentizaci, oprávnění nebo práci s citlivými daty, výkonnost měřitelnou zátěží a použitelnost uživatelské rozhraní. Zvolený projekt je záměrně omezen na doménovou logiku, takže sada Q metrik sleduje funkční korektnost, kvalitu testů a udržitelnost.

Funkční korektnost (Q1–Q2)

Tabulka 3 shrnuje metriky funkční korektnosti.

Tabulka 3: Metriky funkční korektnosti Q1–Q2

Kód	Co měří	Jak se měří	Exit kritérium
Q1	Kompatibilita s API kontraktem	Referenční typy se importují a zkompilují proti agentovu kódu (<code>tsc</code>). Výsledek je binární podle toho, zda typy odpovídají kontraktu.	match
Q2	Úspěšnost referenčních testů	Referenční testy se spouštějí nástrojem Vitest proti agentovu kódu. Výsledek je počet úspěšných testů z 42.	42/42

Q1 ověřuje, zda implementace odpovídá API kontraktu. Pokud implementace API kontrakt nesplňuje, referenční testy Q2 se nedají spustit. Splnění Q1 ale samo neznamená, že se implementace chová správně. To ověřují až referenční testy Q2.

Případová studie rozlišuje dva druhy testů. *Referenční testy* (měřené metrikou Q2) jsou odvozené ze specifikace metodou TDD (Mathews & Nagappan, 2024) a spouštěné po běhu samostatným skriptem, takže pro agenta jsou neviditelné. *Agentovy vlastní testy* píše agent během běhu a jsou viditelné i spustitelné agentem. Vztahuje se k nim metrika P5. Referenční testy slouží jako externí oracle, tedy zdroj očekávaných výsledků, který agent nemůže přepsat (sekce 2.2.3).

Kvalita testů (Q3–Q4)

Tabulka 4 shrnuje metriky kvality testů.

Tabulka 4: Metriky kvality testů Q3–Q4

Kód	Co měří	Jak se měří	Exit kritérium
Q3	Schopnost agentových testů detekovat chyby	Nástroj Stryker spouští mutation testing nad agentovými testy a zdrojovým kódem. Výsledkem je mutation score, tedy podíl mutovaných variant programu, které testy odhalí.	$\geq 80\%$
Q4	Pokrytí acceptance criteria agentovými testy ¹	Specifikace obsahuje 25 acceptance criteria. Hodnocení určí, kolik z nich má odpovídající test v agentově testovací sadě. Výsledek má tvar N/25.	25/25

Kvalita kódu (Q5–Q8)

Tabulka 5 shrnuje metriky kvality kódu.

Tabulka 5: Metriky kvality kódu Q5–Q8

Kód	Co měří	Jak se měří	Exit kritérium
Q5	Lint warnings + errors	ESLint s fixní konfigurací (recommended + strict TypeScript rules). Konfigurace je součástí experimentální infrastruktury, ne agentova repo, takže ji agent nemůže měnit. Výstupem je celkový počet varování a chyb linteru.	0
Q6	Typové chyby	<code>tsc --noEmit</code> ve strict mode. Doplnuje počet explicitních <code>any</code> ve zdrojových souborech (grep v <code>src/**/*.ts</code>) jako proxy pro obcházení typového systému.	0
Q7	Cyklomatická složitost	ESLint <code>complexity</code> rule reportuje funkce s cyklomatickou složitostí nad limitem 10. Výstupem je počet porušení tohoto limitu.	0
Q8	Designová kvalita kódu	Hodnocení posuzuje pojmenování, oddělení zodpovědností, idiomatický TypeScript, kvalitu dokumentace a zbytečnou komplexitu. Celkové skóre je minimum dílčích dimenzí, aby silná dimenze nemaskovala slabou.	$\geq 2/3$

3.2.3 Metriky zdrojů (E1–E3)

Metriky zdrojů zachycují, kolik běh agenta spotřeboval tokenů a času a zda během něj došlo ke kompakci kontextu. Na rozdíl od procesních a produktových metrik nemají E1–E3 exit kritérium. Jejich hodnoty slouží k porovnání spotřeby zdrojů a stability kontextu během běhu mezi iteracemi.

Tato skupina metrik převádí zdrojovou dimenzi z teoretických východisek do agentního prostředí. Lidské úsilí zde není přímo pozorovaným vstupem, proto sledujeme dostupné provozní ukazatele. Jde o spotřebované tokeny, reálný čas běhu a kompakce kontextového okna.

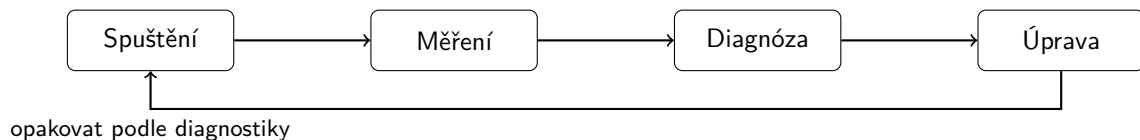
¹Na rozdíl od Q2, která ověřuje chování výsledné implementace, Q4 sleduje, zda agentovy vlastní testy pokrývají požadovaná acceptance criteria.

Tabulka 6: Metriky zdrojů E1–E3

Kód	Co měří	Jak se měří
E1	Spotřeba tokenů	Tři složky z exportu session v tisících: maximální vstup na jednom kroku, součet výstupních tokenů a součet cache tokenů.
E2	Trvání běhu	Reálný čas běhu v minutách od spuštění agenta po poslední commit.
E3	Počet kompakcí	Počet kompakcí kontextu během běhu, čtený přímo z OpenCode (<code>time_compacting</code>). Kompakce indikuje, že agent vyčerpал kontextové okno a prostředí muselo obsah zhustit. Metrika nemá exit kritérium.

3.3 Procedura jedné iterace

Sekce 3.1.1 vymezila rozdíl mezi pilotní a ablační fází. Tato část popisuje operační kroky, které se provedou u jednoho běhu agenta. V pilotní fázi na sebe navazují čtyři kroky: spuštění běhu, měření, diagnóza a úprava instrukcí pro další běh (obr. 2). Pokud další běh vede k regresí, navazující úprava vychází z poslední úspěšné verze instrukcí. V ablační fázi se používá stejné spuštění a měření, ale bez průběžné úpravy téže varianty instrukcí.



Obrázek 2: Iterativní cyklus pilotní fáze.

3.3.1 Spuštění, záznam a vyhodnocení běhu

Každý běh zakládá samostatné experimentální prostředí pro jednu verzi instrukcí a produkuje strukturovaný balík záznamů, který umožňuje zpětně rekonstruovat naměřené hodnoty i chování agenta. Pro orientaci v následujícím popisu ukazuje výpis 1 zjednodušenou strukturu relevantních částí repozitáře.

Výpis 1: Struktura experimentálních materiálů

```

experiments/                                # měřicí infrastruktura, běhy a reference
  infra/                                    # měřicí infrastruktura a vstupy
    inputs/                                # build.md, AGENTS.md a specifikace zadání
    scripts/ts/                             # skripty pro spuštění a vyhodnocení
      new-run.ts                             # založení a spuštění běhu
      analyze-run.ts                         # extrakce deterministických metrik
      judge.ts                               # LLM-as-judge hodnocení
    judge/                                  # hodnoticí schémata
  runs/                                    # výstupy jednotlivých běhů
    pilot-rN/
      transcript.json                        # úplný záznam session
      FINDINGS.md                          # naměřené hodnoty a popis chování

```

reference/

referenční implementace a testy

Spuštění. `new-run.ts` vytvoří pro daný běh lokální adresář `experiments/runs/runName/` a soukromý GitHub repozitář `bp-runName`. Do adresáře zkopíruje zvolenou verzi `AGENTS.md`, fixní systém prompt `build.md`, konfiguraci OpenCode a automatizační plugin. Po inicializačním commitu vytvoří v GitHubu Issue #1 se specifikací projektu a spustí agenta v Docker containeru. Agent pracuje v adresáři běhu jako ve svém pracovním prostoru a dostává úkol implementovat Issue #1 podle `AGENTS.md`. Po ukončení běhu skript uloží identifikátor session, exit code a export session do `transcript.json`, který obsahuje úplný záznam tool calls, vstupů a výstupů agenta.

Sběr metrik. Po skončení běhu se spouštějí dva vyhodnocovací skripty. `analyze-run.ts` extrahuje deterministické metriky z git historie, GitHub API a transcriptu (P1–P5, Q1–Q3, Q5–Q7, E1–E3). Skript `judge.ts` spouští LLM-as-judge (GLM-5) pro judge-based metriky (P6–P8, Q4, Q8) podle hodnoticích schémat uložených v repozitáři práce. Souhrnným výstupem je `FINDINGS.md` s tabulkou metrik a faktickým popisem chování agenta.

Auditovatelný balík. Každý běh má vlastní adresář `experiments/runs/pilot-rN/` obsahující transcript, naměřené metriky a `FINDINGS.md`. K iteraci jako celku patří navíc `DIAGNOSIS.md` s analytickým výstupem (sekce 3.3.2) a záznam v changelogu zdůvodňující úpravu `AGENTS.md` pro další běh. Tabulka 7 shrnuje strukturu balíku.

Tabulka 7: Záznamy pořízené pro jeden běh a jednu iteraci

Soubor	Obsah
<code>transcript.json</code>	úplný záznam session (tool calls, vstup, výstup)
<code>FINDINGS.md</code>	tabulka metrik a faktický popis chování
<code>DIAGNOSIS.md</code>	klasifikace příčin a návrh změn
<code>changelog/rN-rN+1.md</code>	zdůvodnění úprav <code>AGENTS.md</code> (bylo / je / pozorování / diagnóza / odůvodnění)

Doplňující materiály včetně operační procedury, šablon `FINDINGS.md` a `DIAGNOSIS.md`, verzí `AGENTS.md`, hodnoticích schémat, transcriptů session, surových výstupů nástrojů a plného znění specifikace jsou uloženy v repozitáři práce. Přímé odkazy na stav zafixovaný tagem `thesis-final` uvádí příloha B.

3.3.2 Diagnostika a úprava instrukcí

Diagnostika převádí naměřené odchylky na vysvětlení, proč agent selhal nebo odbočil od očekávaného postupu. Z diagnózy potom vychází konkrétní úprava `AGENTS.md`. Vstupem je `FINDINGS.md` z předchozího běhu (sekce 3.3.1), výstupem `DIAGNOSIS.md`, upravená `AGENTS.md` pro příští běh a záznam v changelogu.

Diagnostické rámce. Diagnóza identifikuje, kde a proč agent nedodržel očekávané chování.

Opírá se o pět rámců, z nichž každý klade jinou diagnostickou otázku (tabulka 8).

Tabulka 8: Diagnostické rámce a otázky, které kladou

Rámec	Diagnostická otázka
Mao et al. (2025)	Jsou komponenty instrukce přítomné a ve správném pořadí?
Hassan et al. (2025)	Vyvažuje obsah cíl (briefing), postup (loop) a omezení (mentor)?
Razavi and Fard (2025), Breunig (2025)	Proč agent instrukci opakovaně ignoruje a jak ji přestrukturovat?
Lulla et al. (2026)	Které typy obsahu reálně přispívají k chování? *
Filter X. Li et al. (2026)	Kdyby tento řádek chyběl, udělal by agent neočividnou chybu?

* Lulla měří přítomnost celého souboru, ne jednotlivé typy obsahu, proto se rámec používá interpretativně.

Postup diagnózy. Z tabulky metrik se vyberou nesplněné nebo hraniční hodnoty, dohledají se faktické projevy ve `FINDINGS.md` a v artefaktech (git log, issues, PR, transcript), aby bylo možné odlišit jednorázové selhání běhu od opakujícího se problému instrukcí. Identifikovaná příčina se klasifikuje podle tabulky 9, která mapuje typ příčiny na primární rámec a typ opravy. Tabulka sjednocuje zápis diagnózy napříč iteracemi.

Tabulka 9: Mapování klasifikace příčiny na diagnostický rámec a typ opravy

Klasifikace příčiny	Primární rámec	Typ opravy
Chybějící / příliš obecná složka	Mao	Doplnění / zpřesnění
Nevyvážený obsah	Hassan	Přestrukturování
Opakovaně ignorovaná instrukce	Razavi, Breunig	Přestrukturování + verifikace
Redundantní obsah	Lulla + filter Li	Odebrání

Úprava. Na základě diagnózy se upraví `AGENTS.md`. Každá změna odpovídá jednomu identifikovanému problému a je zaznamenána v changelogu ve formátu *bylo / je / pozorování / diagnóza / odůvodnění*. Záznam umožňuje zpětně dohledat, proč daná úprava vznikla a co měla opravit. Při úpravách má přednost zpřesnění nebo přestrukturování existující instrukce před pouhým přidáváním dalšího textu.

Úpravy instrukcí se v této studii neprovádějí automatickou optimalizací promptu. Ruční diagnostický postup je zvolen proto, aby každá změna měla dohledatelnou vazbu na naměřenou odchylku a konkrétní problém v chování agenta.

3.4 Přehledová tabulka

Tabulka 10 shrnuje všechny metriky na jednom místě. Sběr se liší podle typu metriky. U deterministické metriky sloupec *Nástroj* uvádí konkrétní automatizovaný nástroj, zatímco u judge-based metriky uvádí LLM-as-judge. Sloupec *Exit kritérium* uvádí podmínku úspěchu pro procesní metriky (P1–P8) a produktové metriky (Q1–Q8). Deskriptivní indikátory bez prahu, tedy celá zdrojová skupina E1–E3, mají v tomto sloupci pomlčku a slouží jen k porovnání mezi běhy.

Tabulka 10: Sada metrik podle kódu, měřené vlastnosti, nástroje a exit kritéria

Kód	Metrika	Nástroj	Exit kritérium
<i>Procesní (P): jak agent pracuje</i>			
P1	Issues before code	git, GitHub API	kontrola projde
P2	Branch per issue	git, GitHub API	kontrola projde
P3	Test-first commits	git log	kontrola projde
P4	PRs linked to issues	GitHub API	kontrola projde
P5	No existing test modifications	git diff	kontrola projde
P6	Kvalita zpráv commitů	LLM-as-judge (GLM-5)	$\geq 2/3$
P7	Kvalita popisů issue	LLM-as-judge (GLM-5)	$\geq 2/3$
P8	Kvalita popisů PR	LLM-as-judge (GLM-5)	$\geq 2/3$
<i>Produktové (Q): co agent vyrobil</i>			
Q1	API contract match	tsc (import + typecheck)	match
Q2	Úspěšnost ref. testů	Vitest (42 testů)	42/42
Q3	Mutation score	Stryker	$\geq 80\%$
Q4	AC coverage agentových testů (25 krit.)	LLM-as-judge (GLM-5)	25/25
Q5	Lint warnings	ESLint	0
Q6	Typecheck errors	tsc --noEmit	0
Q7	Cyklomatická složitost	ESLint (complexity)	0
Q8	Design quality	LLM-as-judge (GLM-5)	$\geq 2/3$
<i>Zdroje (E): za jakou cenu</i>			
E1	Vstup / výstup / Σ cache (tis.) z exportu	OpenCode export	—
E2	Trvání (minuty)	session timestamps	—
E3	Počet kompakcí kontextu	OpenCode (time_compacting)	—

3.5 Omezení a validita

Případová studie nese omezení daná zvolenou metodikou. Jejich dopad lze popsat přes konstruktovou, interní a externí validitu podle rámce pro případové studie (Runeson & Höst, 2009).

Konstruktová validita. Sada metrik je autorskou kombinací procesních, produktových a zdrojových ukazatelů. Nejde o ověřený standard pro hodnocení agentů, ale o návrh operacionalizovaný pro tuto případovou studii. Část metrik (P6, P7, P8, Q4, Q8) navíc závisí na LLM-as-judge hodnocení. Tyto metriky zachycují vlastnosti, které nelze spolehlivě extrahovat deterministickým skriptem, ale jejich spolehlivost nebyla validována proti lidským hodnotitelům. Proto jsou interpretovány jako podpůrné judge-based indikátory. Hlavní oporu vyhodnocení tvoří deterministické metriky a auditovatelné artefakty.

Interní validita. Sílu závěrů omezuje nedeterminismus LLM. Stejně instrukce mohou v různých bězích vést k odlišnému chování agenta (Razavi & Fard, 2025). Pilotní běhy proto slouží k postupnému ladění instrukcí, ne k samostatnému dokazování účinku každé jednotlivé změny. Ablací fáze toto omezení zmírňuje dvěma nezávislými běhy pro každou variantu, ale ani tak neposkytuje statistickou sílu. Závěry z ablací proto mají indikativní povahu. Dalším omezením je role autora při diagnostice a úpravě instrukcí. Riziko jednostranné interpretace snižuje pevný diagnostický postup, changelog změn, oddělení měřicí infrastruktury od agentova prostředí a převaha deterministických metrik tam, kde je to možné.

Externí validita. Případová studie na jednom projektu, jednom modelu a jednom agentním nástroji neumožňuje statistickou generalizaci. Ve smyslu analytické generalizace (R. K. Yin, 2018) může jeden případ ukázat principy postupu, ne obecné zákonitosti pro všechny agenty a projekty. Zvolený projekt má deterministickou doménovou logiku a nemusí reprezentovat úlohy s uživatelským rozhraním, strojovým učením nebo nedeterministickými výstupy. Z těchto podmínek lze zobecnit hlavně způsob práce s metrikami, tedy jak běh zaznamenat, vyhodnotit a použít výsledky k úpravě instrukcí. Nelze z nich odvodit, že stejné hodnoty metrik nebo stejné instrukce budou fungovat u jiného modelu, nástroje nebo projektu.

Výzkum nezahrnuje lidské účastníky. Data (git log, metriky, session transcripts), zdrojový kód i experimentální artefakty jsou veřejně dostupné.

4. Praktická část

Tato kapitola popisuje provedení případové studie. Specifikace projektu, referenční testy a experimentální nastavení jsou popsány v kapitole 3. Struktura kapitoly sleduje chronologii studie. Nejprve popisuje výchozí instrukce použité v prvním běhu, pak pilotní fázi kde se instrukce opakovaně upravují na základě naměřených metrik, a nakonec ablace, kde z fungující sady systematicky odebíráme jednotlivé části a měříme dopad.

4.1 Konstrukce výchozích instrukcí

Metodika vymezuje výchozí AGENTS.md jako první variantu instrukcí pro první pilotní běh. Tato část ukazuje její konkrétní podobu. Výchozí varianta spojuje tři vrstvy, které vyplynuly z teoretických východisek. Jde o vymezení úkolu, procedurální pracovní postup a verifikační kroky kvality. Výsledný soubor není vyčerpávající manuál k projektu. Má 53 řádků a přibližně 350 slov, aby agent dostal konkrétní oporu pro práci, ale zároveň nebyl zatížen nadbytečným kontextem.

Struktura výchozí varianty převádí komponenty instrukcí popsané Mao et al. (2025) na sedm sekcí. Jsou to Role, Goal, Specification, Environment, Process, Package Quality a Constraints. Sekce Specification váže práci agenta na Issue #1, acceptance criteria a API kontrakt. Process určuje postup přes GitHub issues, větve, testy, implementaci a pull requesty. Package Quality doplňuje požadavky na modularitu, striktní TypeScript, dokumentaci a čisté veřejné API, protože tyto vlastnosti nejsou samy o sobě vynutitelné doménovou specifikací. Constraints zachycují zakázané vzorce práce, například slučování více issues do jedné větve, úpravy testů podle implementace a přepis git historie.

Tabulka 11 ukazuje, jak sekce výchozího AGENTS.md odpovídají měřeným vlastnostem. Plné znění výchozího AGENTS.md je uvedeno hned za tabulkou.

Tabulka 11: Mapování sekcí výchozího AGENTS.md na metriky

Cíl	Sekce AGENTS.md	Metriky
Spec-first	Specification, Process krok 2	P1
Strukturovaný pracovní postup	Process kroky 1–4	P2, P3, P4
TDD	Process krok 2, Constraints	P3, P5
Modulární kód	Package Quality	Q5, Q7, Q8
API kompatibilita	Specification (API Contract)	Q1
Funkční korektnost	Specification (AC)	Q2, Q4

Výpis 2: Výchozí AGENTS.md pro pilot-r1

```
# AGENTS.md -- Dunning System

## Role

You are a senior TypeScript developer building a production-grade npm package. You prioritize
clean architecture, spec compliance, and test quality.

## Goal

Implement the dunning system (billing reminder state machine) specified in GitHub Issue #1.
Deliver a modular, documented, publishable TypeScript package.

## Specification

Read Issue #1 completely before writing any code. It contains:
- **Acceptance Criteria** -- every one must be satisfied and tested
- **API Contract** -- exact TypeScript signatures to export
- **Domain Glossary** -- use these terms in code and documentation
- **Out of Scope** -- do not implement

All design decisions must trace back to the spec. Do not invent requirements.

## Environment

- Runtime: Node.js, TypeScript (strict mode, ESM)
- Test runner: Vitest
- Build: tsc
- Tools: GitHub CLI ('gh'), Git

## Process

Decompose the spec into focused GitHub issues before writing code. Work through them
sequentially:

1. Create issues -- one concern per issue, starting with project setup
2. For each issue: branch from main, write tests first, implement, PR with "Closes #N", merge
3. Use conventional commits: 'test:' for tests, 'feat:' for implementation, 'docs:' for
documentation
4. Run 'tsc --noEmit' before every PR -- no type errors allowed

## Package Quality

Structure the code as a production-grade npm package:

- **Modular architecture** -- separate files for types, business logic, utilities, and public API
re-exports. No single-file implementations.
- **Strict TypeScript** -- no 'any', explicit return types on public API, 'readonly' where
applicable.
- **Documentation** -- JSDoc on all exported functions and types. Inline comments for non-obvious
domain logic (e.g., business day calculations, escalation thresholds).
- **Clean API surface** -- 'src/index.ts' re-exports only the public API. Internal modules are
not exposed.

## Constraints

- Never combine multiple issues into one branch.
- Never implement without a failing test first.
- Never modify a test to match your implementation. Tests encode the spec -- fix the code, not
the test.
- Never rewrite git history (no amend, squash, rebase, force-push).
- Do not add dependencies beyond the dev toolchain (vitest, typescript).
```

4.2 Pilotní fáze

Pilotní fáze začíná během r1 s výchozími instrukcemi. Další podsekcce popisují jednotlivé běhy, naměřené hodnoty, diagnózu a úpravu instrukcí pro navazující iteraci.

4.2.1 Pilot-r1: výchozí běh

První běh s výchozími instrukcemi (verze v repozitáři práce), agent MiniMax-M2.5 přes OpenCode, proběhl v jedné session bez restartů. Tabulka 12 shrnuje naměřené metriky.

Tabulka 12: Pilot-r1: naměřené metriky

Kód	Metrika	Hodnota	Kritérium	Splněno?
P1	Issues before code	✓	splněno	✓
P2	Branch per issue	×	branches $\geq$ issues	×
P3	Test-first commits	×	test: před feat:	×
P4	PRs linked to issues	✓	všechny PR	✓
P5	Testy nezměněny	×	0 změn	×
P6	Kvalita zpráv commitů	n/a	$\geq 2/3$	n/a
P7	Kvalita popisů issue	3/3	$\geq 2/3$	✓
P8	Kvalita popisů PR	2/3	$\geq 2/3$	✓
Q1	API contract match	match	match	✓
Q2	Úspěšnost ref. testů	39/42	42/42	×
Q3	Mutation score	84 %	$\geq 80 \%$	✓
Q4	AC coverage (agentovy testy)	23/25	25/25	×
Q5	Lint warnings	2	0	×
Q6	Typecheck errors	0	0	✓
Q7	Porušení složitosti	2	0	×
Q8	Design quality	1/3	$\geq 2/3$	×
E1	Vstup / výstup / Σ cache (tis.)	115/60/11528	n/a	záznam
E2	Trvání	33 min	n/a	záznam
E3	Počet kompakcí	0	n/a	záznam

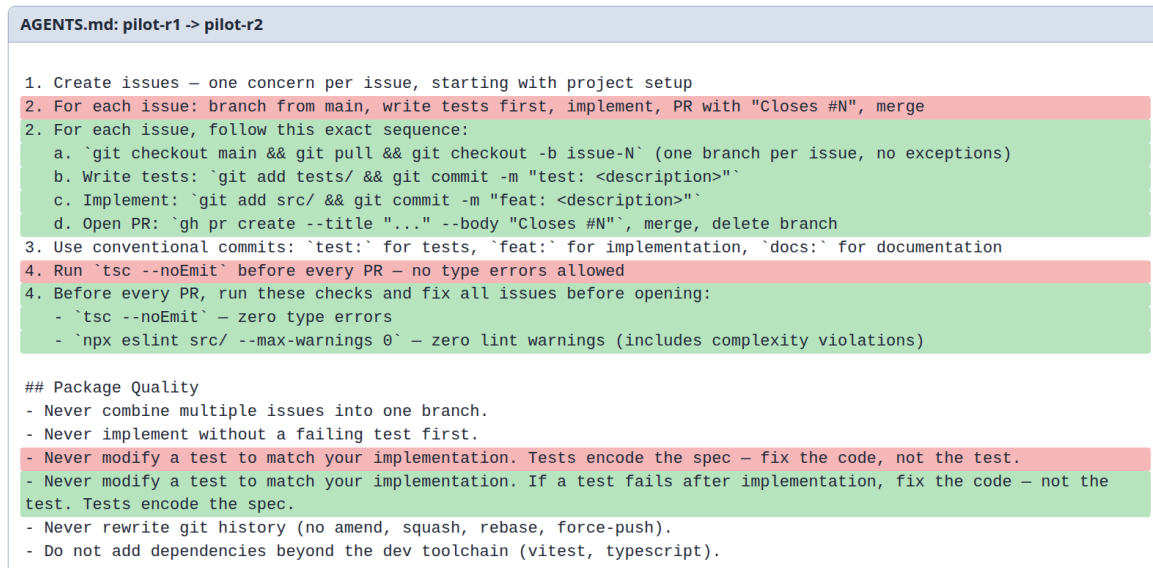
Diagnostika chování

Tabulka 13: Pilot-r1: nálezy a jejich příčiny

Metriky	Pozorované chování	Příčina
P2	Issues seskupeny do méně větví (4 větve / 11 issues)	Instrukce neuvádí pravidlo „1 issue = 1 větev“ a chybí procedurální páka
P3	Implementace předcházela testům napříč všemi větvemi	Žádné explicitní pravidlo ani kontrola pořadí test/kód
P5	V jedné větvi upraven existující test	Žádná hard-rule pro existující testy
Q5, Q7	2 varování linteru, 2 funkce nad prahem složitosti	Package Quality nevynucuje spuštění lintu ani kontrolu složitosti
Q8	Chybějící JSDoc na veřejných funkcích z API kontraktu	Deklarativní požadavek bez verifikačního kroku
Q2	3 z 42 referenčních testů selhaly (pause/resume, eskalace)	Implementační rozhodnutí v doménové logice mimo dosah výchozích instrukcí
Q4	Agentovy testy pokryly 23 z 25 AC (chyběly testy pro custom holiday calendar a edge case)	Process (TDD) nevyžaduje kontrolu, že každé AC má pokrývající test

Závěry a intervence pro r2

Společný pattern napříč P2, P3, P5 a Q5/Q7 spočívá v tom, že instrukce nese „co“ má platit bez konkrétního „jak“ a bez kontroly (viz tabulka). Selhání Q2 a Q4 jsou jiného typu. Implementační rozhodnutí v pause/resume logice a chybějící AC pokrytí v doménové logice, na která úroveň výchozích instrukcí nedosahuje. Pro r2 doplňujeme procedurální páku ke každému selhanému procesnímu pravidlu (příkaz pro vytvoření větve, oddělené staging testů a kódu, korektivní pravidlo pro selhávající test) a kontrolu lintu před otevřením pull requestu. Q8 (JSDoc) a Q4 (AC pokrytí) ponecháváme jako deklarativní pravidla v Package Quality, abychom v r2 vyhodnotili, zda zopakovaná deklarace bez verifikace zabere. Pokud ne, přesuneme je v r3 do verifikačního kroku (obrázek 3).



Obrázek 3: Vizuální diff změn AGENTS.md mezi pilot-r1 a pilot-r2

4.2.2 Pilot-r2

Druhý běh ověřoval čtyři změny instrukcí. Tabulka 14 ukazuje metriky kde došlo ke změně.

Tabulka 14: Pilot-r2: vývoj sledovaných metrik oproti r1

Kód	Metrika	r1	r2	Trend
<i>Přetrvávající selhání (intervence cílila, hodnota beze změny):</i>				
P2	Branch per issue	×	×	přetrvává
P3	Test-first commits	×	×	přetrvává
Q8	Design quality	1/3	1/3	přetrvává
<i>Změna proti r1:</i>				
Q2	Úspěšnost ref. testů	39/42	32/42	regrese
Q3	Mutation score	84 %	68 %	regrese
P5	Testy nezměněny	×	✓	zlepšení
Q5	Lint warnings	2	1	zlepšení
Q7	Porušení složitosti	2	0	zlepšení
E1	Vstup / výstup / Σ cache (tis.)	115/60/11528	76/41/3196	záznam
E2	Trvání	33 min	37 min	záznam
E3	Počet kompakcí	0	0	záznam

Diagnostika chování

Tabulka 15: Pilot-r2: nálezy a jejich příčiny

Metriky	Pozorované chování	Příčina
P2	Issues agregovány do menšího počtu větví (7 issues / 3 větve)	Procedura neměla pravidlo „1 issue = 1 větev“
P3	Test+kód v jednom commitu s prefixem <code>test:</code> , žádný <code>feat:</code> commit	Formální dodržení příkazu bez naplnění jeho záměru (oddělit testovací a implementační commity)
Q8	JSDoc neaplikován	Deklarativní pravidlo bez verifikační opory
Q2, Q3	Pokles kvality testovací sady (více testů, méně zachycených mutantů)	Hromadné psaní testů ze specifikace místo iterativního test-fix cyklu

Závěry a intervence pro r3

V obou procesních pravidlech (P2, P3) agent příkaz dodržel formálně, ne podle záměru. Jde o typický pattern u deklarativních instrukcí bez verifikační opory (viz tabulka). Stejný posun se projevil i na produktové straně, pokles Q2 a Q3 ukazuje, že hromadná write strategie produkuje testy, které hůř zachytí chyby v kódu než iterativní test-fix cyklus. Pro r3 přesouváme tři instrukce z roviny příkazu do verifikačního kroku. Přidává se explicitní výpis otevřených issues před výběrem práce, verifikační `git log` mezi commity testů a kódu a přesun požadavku na JSDoc do verifikační části před PR. Na Q2/Q3 necílíme přímo, ale očekáváme nepřímý efekt přes test-fix cyklus, který verifikační `git log` mezi commity testů a kódu strukturálně podporuje (oproti hromadnému psaní testů ze specifikace) (obrázek 4).

AGENTS.md: pilot-r2 -> pilot-r3	
1. Create issues – one concern per issue, starting with project setup	
2. For each issue, follow this exact sequence:	
a. <code>`git checkout main && git pull && git checkout -b issue-N`</code> (one branch per issue, no exceptions)	
a. Pick one open issue: <code>`gh issue list --state open`</code> , then <code>`git checkout main && git pull && git checkout -b issue-N`</code>	
b. Write tests: <code>`git add tests/ && git commit -m "test: <description>"`</code>	
c. Implement: <code>`git add src/ && git commit -m "feat: <description>"`</code>	
d. Open PR: <code>`gh pr create --title "... " --body "Closes #N"`</code> , merge, delete branch	
c. Verify: run <code>`git log --oneline -3`</code> and confirm the test: commit exists before proceeding	
d. Implement: <code>`git add src/ && git commit -m "feat: <description>"`</code>	
e. Open PR: <code>`gh pr create --title "... " --body "Closes #N"`</code> , merge, delete branch	
3. Use conventional commits: <code>`test:`</code> for tests, <code>`feat:`</code> for implementation, <code>`docs:`</code> for documentation	
4. Before every PR, run these checks and fix all issues before opening:	
- Modular architecture – separate files for types, business logic, utilities, and public API re-exports. No single-file implementations.	
- Strict TypeScript – no <code>`any`</code> , explicit return types on public API, <code>`readonly`</code> where applicable.	
- Documentation – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds).	
- Documentation – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds). This is enforced in Constraints.	
- Clean API surface – <code>`src/index.ts`</code> re-exports only the public API. Internal modules are not exposed.	
- Never implement without a failing test first.	
- Never modify a test to match your implementation. If a test fails after implementation, fix the code – not the test. Tests encode the spec.	
- Before every PR, verify JSDoc on every exported function in <code>`src/index.ts`</code> . If any export lacks JSDoc, add it before opening the PR.	
- Never rewrite git history (no amend, squash, rebase, force-push).	
- Do not add dependencies beyond the dev toolchain (vitest, typescript).	

Obrázek 4: Vizuální diff změn AGENTS.md mezi pilot-r2 a pilot-r3

4.2.3 Pilot-r3

Třetí běh ověřoval tři změny instrukcí z r2, tedy verifikační příkaz po commitu s testy, výběr issue přes `gh issue list` a přesun požadavku na JSDoc do verifikačního kroku před pull requestem. Tabulka 16 ukazuje metriky s výraznou změnou.

Tabulka 16: Pilot-r3: metriky se změnou oproti r2

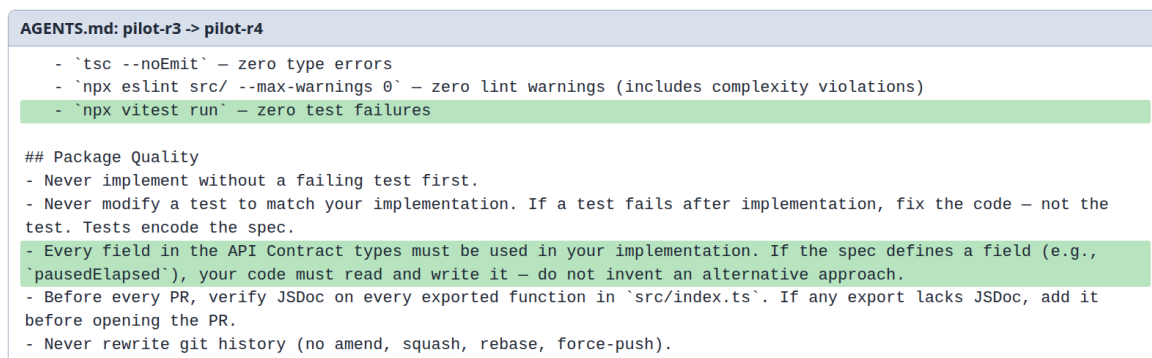
Kód	Metrika	r2	r3	Trend
P2	Branch per issue	×	✓	zlepšení
P3	Test-first commits	×	✓	zlepšení
P6	Kvalita zpráv commitů	2/3	3/3	zlepšení
P7	Kvalita popisů issue	2/3	3/3	zlepšení
Q2	Úspěšnost ref. testů	32/42	41/42	zlepšení
Q3	Mutation score	68 %	71 %	zlepšení
Q7	Porušení složitosti	0	1	regrese
Q8	Design quality	1/3	3/3	zlepšení
E1	Vstup / výstup / Σ cache (tis.)	76/41/3196	62/30/4770	záznam
E2	Trvání	37 min	25 min	záznam
E3	Počet kompakcí	0	0	záznam

Diagnostika chování

¹ Tři změny instrukcí z r2 zabraly. Agent dodržel celý procesní checklist (P1–P5 = 5/5) a doplnil JSDoc dokumentaci (Q8 = 3/3). Zbývá jediný nález. Agent zvolil vlastní implementaci pause/resume (posunutí `stateEnteredAt`) místo pole `pausedElapsed` z API kontraktu. Přístup fungoval pro jedno pozastavení, ale selhal při opakovaném (Q2 = 41/42). Specifikace přitom byla jednoznačná, acceptance criteria vyžadovala zachování uplynulého času a API kontrakt poskytoval explicitní pole.

Závěry a intervence pro r4

Tři změny z r2 ukazují, že přesun deklarativních pravidel do verifikačního kroku funguje. Zbývajících nález naznačuje, že explicitní opora v typech sama o sobě nestačí, pokud agent učiní vlastní implementační rozhodnutí. Mírná regrese Q7 (0→1, pod exit kritériem 0) je v pásmu variability mezi běhy. Samostatná intervence proto není prioritou r4, který se zaměřuje na selhání Q2. Pro r4 doplňujeme do verifikační části před PR příkaz `npx vitest run`, aby agent před otevřením pull requestu ověřil průchod vlastních testů, a do Constraints pravidlo, že každé pole v API kontraktu musí být v implementaci použito (Breunig, 2025) (obrázek 5).



Obrázek 5: Vizuální diff změn AGENTS.md mezi pilot-r3 a pilot-r4

4.2.4 Pilot-r4

Čtvrtý běh ověřoval dvě úpravy. První bylo přidání `npx vitest run` do verifikační části před PR, druhé pravidlo o dodržení API kontraktu (každé pole v typech musí být použito v implementaci). Tabulka 17 ukazuje metriky s nejvýraznější změnou.

¹V r3 uvádíme diagnostiku jako prose, protože iterace identifikuje jediný informativní nález. Samostatná findings tabulka by duplikovala obklopující text.

Tabulka 17: Pilot-r4: metriky se změnou oproti r3

Kód	Metrika	r3	r4	Trend
P2	Branch per issue	✓	×	regrese
P5	Testy nezměněny	✓	×	regrese
Q2	Úspěšnost ref. testů	41/42	39/42	regrese
Q3	Mutation score	71 %	n/a	n/a
Q8	Design quality	3/3	2/3	regrese
E1	Vstup / výstup / Σ cache (tis.)	62/30/4770	81/36/5996	záznam
E2	Trvání	25 min	26 min	záznam
E3	Počet kompakcí	0	0	záznam

Poznámka: Q3 = n/a, protože agentovy vlastní testy selhávají (timezone bug ve `businessDays`, viz tab. 18). Stryker vyžaduje funkční testovací sadu, mutace nelze odlišit od výchozího selhání testů. Nejde o regresi měřené hodnoty.

Diagnostika chování

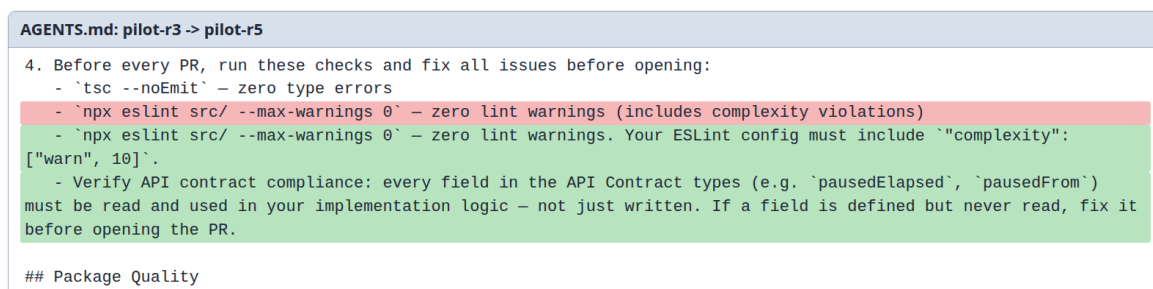
Tabulka 18: Pilot-r4: nálezy a jejich příčiny

Metriky	Pozorované chování	Příčina
P2, P5	Procesní pravidla porušena (žádné větve per issue, modifikované vlastní testy)	Mezi-běhová variabilita modelu (změna konfigurace v Docker prostředí + nedeterminismus). Intervence r3→r4 cílila Q2, ne procesní pravidla
Q2	Pole <code>pausedElapsed</code> opět nepoužito, přibýly chyby v eskalaci v pracovních dnech a na boundary přechodu DUE_SOON	Pravidlo o API kontraktu v Constraints (deklarativní) bez verifikace neúčinné
Q2	Začleněn kód s timezone bugem ve vlastním testu	<code>npx vitest run</code> ve verifikační části před PR nevytulo skutečné úspěšné dokončení testů
Q8	Pokles design quality (3/3→2/3), JSDoc chybí na utility functions (<code>businessDays.ts</code>) a pause/resume logika je nedokumentovaná	Intervence r3→r4 cílila Q2, požadavek na JSDoc z r3 zůstal deklarativní bez kontroly

Závěry a intervence pro r5

Deklarativní pravidlo v Constraints nepřineslo zlepšení a instrukce na této úloze zůstávají citlivé na variabilitu mezi běhy. Mezi r3 a r4 se kromě instrukcí změnila i konfigurace modelu v Docker prostředí (sekce 5.3). Procesní regrese (P2, P5) i kvalitativní pokles (Q2, Q8) proto nelze připsat výhradně dvěma novým řádkům v Constraints. Pro r5 vycházíme z r3 (ne z r4) a cílíme na dvě zbývající selhání verifikačními kroky. Pro Q2 doplňujeme do verifikační části před PR ověření, že každé pole z typů v API kontraktu je v implementaci čteno i zapisováno

(oproti r4, kde pravidlo bylo v Constraints, je verifikace v Process (Breunig, 2025)). Pro Q5/Q7 přidáváme explicitní požadavek na ESLint config zahrnující `complexity: [warn, 10]`, protože v r3 agent spustil eslint s vlastním configem bez tohoto pravidla (obrázek 6).



Obrázek 6: Vizuální diff změn AGENTS.md mezi pilot-r3 a pilot-r5

4.2.5 Pilot-r5: verifikace kontraktu

Pátý běh ověřoval změny zavedené po r4, tedy verifikaci API kontraktu ve verifikační části před PR a ESLint complexity pravidlo.

Tabulka 19: Pilot-r5: metriky se změnou oproti r3

Kód	Metrika	r3	r5	Trend
P1	Issues before code	✓	×	regrese
P2	Branch per issue	✓	×	regrese
P3	Test-first commits	✓	×	regrese
P4	PRs linked	✓	×	regrese
P8	Kvalita popisů PR	3/3	1/3	regrese
Q2	Úspěšnost ref. testů	41/42	38/42	regrese
Q3	Mutation score	71 %	n/a	n/a
Q5	Lint warnings	1	0	zlepšení
Q7	Porušení složitosti	1	0	zlepšení
E1	Vstup / výstup / Σ cache (tis.)	62/30/4770	65/23/3028	záznam
E2	Trvání	25 min	13 min	záznam
E3	Počet kompakcí	0	0	záznam

Poznámka: Q3 = n/a, protože agentovy vlastní testy selhávají dramaticky (4/29 selhalo, chyby v přechodech stavů). Stryker vyžaduje funkční testovací sadu, mutace nelze odlišit od výchozího selhání testů. Stejný mechanismus jako u r4 a Ablace A-2. Nejde o regresí měřené hodnoty.

Diagnostika chování

Tabulka 20: Pilot-r5: nálezy a jejich příčiny

Metriky	Pozorované chování	Příčina
P1, P2, P3, P4, P8	Žádné issues, větve ani PR. Vše v jednom commitu s prefixem feat: po použití inter-ního todowrite místo gh issue create . Kvalita PR 1/3 (chybějící popis a kontext)	Selektivní plnění instrukcí. Agent přečetl celý postup, ale provedl jen závěrečný checklist
Q2	Stále neúplné pause/resume a nové edge cases	Verifikace API kontraktu nezabrala bez současného dodržení procesu

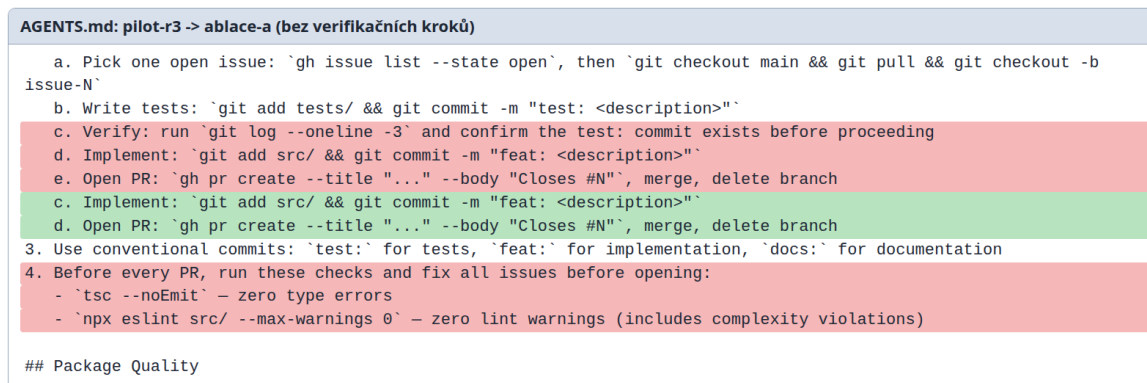
Závěr pilotní fáze

Srovnání r3, r4 a r5 ukazuje, že selektivní plnění není náhodné. R3 celý pracovní postup dodržel ($P1-P5 = 5/5$), r4 částečně ($\sim 3/5$), r5 ignoroval zcela ($\sim 1/5$), přitom instrukce se mezi r3 a r5 lišily minimálně (2 řádky ve verifikační části před PR). Data tak naznačují, že nedeterminismus modelu je pro dodržování vícekrokových sekvencí silnějším faktorem než drobné lokální změny v checklistu. Lokální zlepšení ($Q5, Q7 = 0$) přitom přišlo díky konkrétnímu ESLint pravidlu ve verifikační části před PR, což ukazuje, že jednokrokové verifikační příkazy zafungovaly i při selhání procesní sekvence. Výchozí variantou pro ablace zůstává r3. Ablace testují, které složky fungujících instrukcí ke zlepšení skutečně přispěly. R5 se proto nepoužívá jako výchozí varianta pro ablace. Zachycuje variabilitu dodržení pracovního postupu, ale nesplňuje podmínku fungujícího procesního základu.

4.3 Ablace

Pilotní fáze ukázala, že část zlepšení souvisí s přidáním konkrétních verifikačních kroků. Zbývá proto otázka, které složky výsledných instrukcí jsou pro chování agenta nezbytné a které lze odebrat bez dopadu. Po pilotu zůstaly otevřené dvě konkrétní nejistoty. Nebylo jasné, zda agent potřebuje explicitně předepsané verifikační kroky a zda sekce Package Quality nese samostatný přínos nad rámec pracovního postupu. Ablace proto nejsou generickým odebíráním částí **AGENTS.md**. Každá odpovídá na otázku, která po r1–r5 zůstala neuzavřená.

Existující studie měří efekt souboru **AGENTS.md** převážně jako celku. Účinek jednotlivých složek proto zůstává otevřenou otázkou (Gloaguen et al., 2026; Lulla et al., 2026). Naše ablace tuto mezeru částečně vyplňují na úrovni případové studie. Testujeme dvě ablační varianty a každou spouštíme ve dvou nezávislých bězích, aby bylo možné odlišit shodný směr změny od jednorázového selhání konkrétního běhu.

Obrázek 7: Vizuální diff AGENTS.md mezi pilot-r3 a ablací A.²

4.3.1 Výběr složek pro ablaci

Soubor AGENTS.md obsahuje sedm sekcí. Pro účely ablaci rozlišujeme sekce definující *co* implementovat (Role, Goal, Specification, Environment) a sekce definující *jak* pracovat a jakou kvalitu produkovat (Process, Package Quality, Constraints). Odebrání kontextových sekcí by testovalo jinou otázku než vliv instrukcí na proces a kvalitu. Constraints zde navíc neablujeme, protože jejich efekt se v pilotních bězích částečně překrýval s pracovním postupem v Process. Proto testujeme dvě složky, u nichž zůstala největší nejistota, tedy verifikační kroky v Process a sekci Package Quality.

Ablace A: bez verifikačních kroků

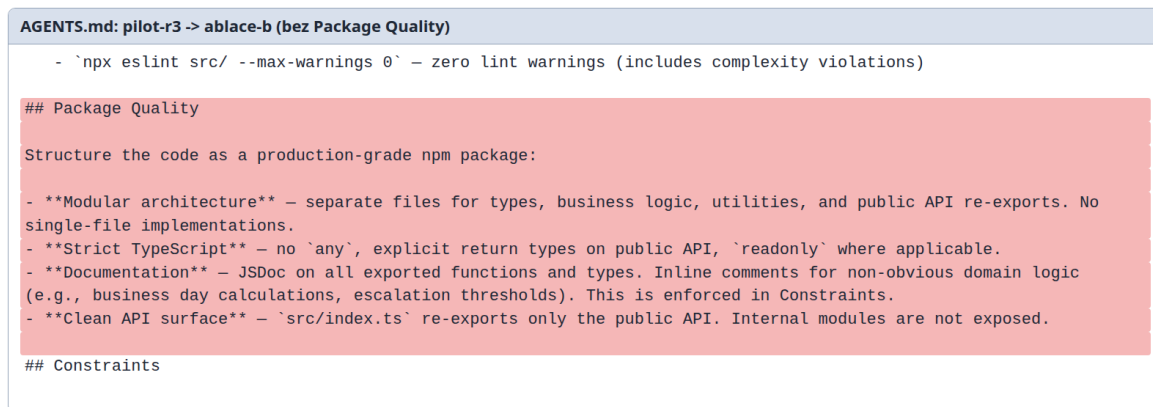
Odebíráme explicitní verifikační příkazy (`tsc --noEmit`, `npx eslint`, `git log --oneline -3`), ale ponecháváme pracovní postup `issue → branch → test → implement → PR` (obrázek 7). Cílem je ověřit, zda agent tyto verifikační kroky spustí i bez instrukce.

Očekávání: Pokud agent typovou kontrolu a lint spustí i bez instrukce, verifikační kroky jsou redundantní. Pokud ne, jde o klíčový mechanismus jejich účinku.

Ablace B: bez sekce Package Quality

Odebíráme celou sekci Package Quality (modulární architektura, strict TypeScript, JSDoc dokumentace, čisté veřejné API). Process a Constraints zůstanou beze změny (obrázek 8).

²Zelené řádky v diffu neobsahují nový text. Vznikly přečíslováním kroků po odebrání kroku c (Verify).



Obrázek 8: Vizuální diff AGENTS.md mezi pilot-r3 a ablací B.

Očekávání: Pokud Q5, Q7 a Q8 zůstanou na úrovni r3, jsou konvenční instrukce převážně redundantní s tréninkem modelu. Pokud se zhorší, explicitní konvence v instrukcích přispívají k celkovému efektu souboru.

4.3.2 Ablace A: bez verifikačních kroků

Tabulka 21 shrnuje výsledky dvou běhů (A-1, A-2) ve srovnání s r3.

Tabulka 21: Ablace A: srovnání s r3 (bez verifikačních kroků)

Kód	Metrika	r3	A-1	A-2
P1	Issues before code	✓	✓	✓
P2	Branch per issue	✓	✓	×
P3	Test-first commits	✓	✓	×
P4	PRs linked to issues	✓	✓	×
P5	Testy nezměněny	✓	×	✓
P6	Kvalita zpráv commitů	3/3	2/3	2/3
P7	Kvalita popisů issue	3/3	2/3	3/3
P8	Kvalita popisů PR	3/3	3/3	1/3
Q1	API contract match	match	match	match
Q2	Úspěšnost ref. testů	41/42	35/42	37/42
Q3	Mutation score	71 %	62 %	n/a [†]
Q5	Lint warnings	1	4	3
Q6	Typecheck errors	0	0	0
Q7	Porušení složitosti	1	4	1
Q8	Design quality	3/3	2/3	2/3
E1	Vstup / výstup / Σ cache (tis.)	62/30/4770	72/32/4558	92/36/8210
E2	Trvání	25 min	39 min	24 min
E3	Počet kompakcí	0	0	0

[†] Q3 v A-2 nelze měřit, protože agentovy vlastní testy selhávají (test očekává stav `REMINDER_1`, kód vrací `GRACE`), Stryker vyžaduje funkční testovací sadu. Bez verifikačních kroků agent odevzdal kód s nefunkčními testy.

Diagnostika chování

Tabulka 21 ukazuje pokles napříč metrikami kvality kódu (Q5, Q7) i funkční korektnosti (Q2, Q3). Bez explicitního příkazu `npx eslint` agent neprovedl kontrolu kvality před odevzdáním kódu, což naznačuje, že verifikační kroky nejsou redundantní. Agent je bez instrukce nespouští sám.

Pokles Q8 (3/3→2/3 v obou bžích) má v každém běhu jiný mechanismus. A-1 obešel typový systém pomocí `undefined as unknown as number` v `getDefaultTimeouts()` místo modelování volitelných hodnot, A-2 produkoval monolitní strukturu s duplicitními helpery (`startOfDay`) a bez modulových hranic. Bez verifikačního kroku zůstávají oba problémy ve výsledném kódu.

Procesní metriky vykazují vzorec známý z pilotních iterací. A-1 dodržel pracovní postup kromě modifikace vlastních testů ($P5 = \times$), A-2 selhal v klíčových procesních pravidlech (P2, P3, P4). Instrukce se mezi běhy nelišily. Variabilita je důsledkem nedeterminismu modelu, ne ablace.

Agent v Ablaci A spustil přibližně o 40 % více testovacích cyklů než v Ablaci B (A-1: 25, A-2: 20, B-1: 20, B-2: 12, měřeno spuštěními `vitest` a `npm test` v logu volání nástrojů). Řetězené spuštění `tsc`, `eslint` a `vitest` v Ablaci B zachycuje chyby dříve.

4.3.3 Ablace B: bez Package Quality

Tabulka 22 shrnuje výsledky dvou běhů (B-1, B-2) ve srovnání s r3.

Tabulka 22: Ablace B: srovnání s r3 (bez sekce Package Quality)

Kód	Metrika	r3	B-1	B-2
P1	Issues before code	✓	✓	✓
P2	Branch per issue	✓	✓	×
P3	Test-first commits	✓	✓	✓
P4	PRs linked to issues	✓	×	✓
P5	Testy nezměněny	✓	×	×
P6	Kvalita zpráv commitů	3/3	3/3	3/3
P7	Kvalita popisů issue	3/3	3/3	3/3
P8	Kvalita popisů PR	3/3	2/3	2/3
Q1	API contract match	match	match	match
Q2	Úspěšnost ref. testů	41/42	37/42	11/42
Q3	Mutation score	71 %	67 %	72 %
Q5	Lint warnings	1	2	2
Q6	Typecheck errors	0	0	0
Q7	Porušení složitosti	1	1	2
Q8	Design quality	3/3	2/3	2/3
E1	Vstup / výstup / Σ cache (tis.)	62/30/4770	78/40/6273	55/23/3604
E2	Trvání	25 min	28 min	21 min
E3	Počet kompakcí	0	0	0

Diagnostika chování

Deterministické metriky kvality kódu (Q5, Q6, Q7) zůstaly na podobné úrovni jako v r3 (tabulka 22). Agent v obou bězích udržel striktní TypeScript bez `any` a JSDoc na hlavním API, přestože instrukce tyto požadavky neobsahovaly. Data naznačují, že tyto základní kódové konvence (typování, idiomatický styl) jsou u tohoto modelu řízeny převážně znalostmi z tréninku.

Q8 kleslo z r3 (3/3) na 2/3 v obou bězích, ale s odlišnými strukturálními mechanismy. B-1 zachoval 3-modulové oddělení (`business-days.ts`, `dunning.ts`, `index.ts`), chyběla mu ale dokumentace na pomocných funkcích. B-2 spadl do jednosouborového monolitu s pomocnými funkcemi pro práci s daty smíchanými s logikou stavového automatu a JSDoc měl jen na hlavním API. Společným vzorcem napříč ablaci B je ztráta JSDoc na utility funkcích, tedy přesně požadavku, který sekce Package Quality explicitně formulovala. Modulární separace mezi dvěma běhy kolísala, což ukazuje, že tato strukturální dimenze je u tohoto modelu citlivější na nedeterminismus než na samotnou ablaci sekce. Závěr stojí na judge-based metrice bez validace proti lidskému hodnocení.

Variabilita mezi B-1 a B-2 v Q2 (37/42 vs. 11/42, mutation score stabilní 67–72 %), v procesních pravidlech (P2: ✓/×, P4: ×/✓, sdílená regrese P8 3/3→2/3) a v P5 (selhání obou běhů, modifikace vlastních testů) nesouvisí s ablací Package Quality, ta neobsahuje instrukce o implementační logice ani procesu. Jde o nedeterminismus modelu. Pokles Q2 v B-2 způsobil chybný business day výpočet a kaskádové selhání eskalačních přechodů, P5 a procesní variabilita se objevují i v r1 a r4 se shodnými instrukcemi. Mechanismus regrese P8 v dostupných artefaktech nezachycen.

4.4 Souhrnné výsledky

Tabulka 23 shrnuje naměřené hodnoty všech metrik (deterministické metriky i judge-based metriky) napříč devíti běhy, tedy pět pilotních iterací a dvě ablační varianty, každou ve dvou nezávislých bězích. Pilotní sloupce ukazují evoluci instrukcí, ablační sloupce ukazují dopad odebrání jednotlivých složek a současně variabilitu mezi opakováním téže varianty. Výchozí variantou pro srovnání je r3, protože jde o poslední variantu, která splnila celý procesní checklist. Sloupec E1 uvádí vstup (max. na kroku včetně `cache` v exportu), výstup a součet `cache` přes kroky, vše v tisících. Stejný přehled generuje skript `summary.ts` do `experiments/runs/SUMMARY.md`.

Tabulka 23: Souhrnné výsledky všech běhů (pilot + ablace)

Metrika	Pilotní fáze					Ablace A		Ablace B	
	r1	r2	r3	r4	r5	A-1	A-2	B-1	B-2
P1 issues before code	✓	✓	✓	✓	×	✓	✓	✓	✓
P2 branch per issue	×	×	✓	×	×	✓	×	✓	×
P3 test-first	×	×	✓	✓	×	✓	×	✓	✓
P4 PRs linked	✓	✓	✓	✓	×	✓	×	×	✓
P5 testy nezměněny	×	✓	✓	×	✓	×	✓	×	×
P6 commit kvalita	n/a	2/3	3/3	3/3	2/3	2/3	2/3	3/3	3/3
P7 issue kvalita	3/3	2/3	3/3	3/3	3/3 [‡]	2/3	3/3	3/3	3/3
P8 PR kvalita	2/3	3/3	3/3	3/3	1/3	3/3	1/3	2/3	2/3
Q1 API contract	match	match	match	match	match	match	match	match	match
Q2 úspěšnost ref. testů	39	32	41	39	38	35	37	37	11
Q3 mutation score	84 %	68 %	71 %	n/a*	n/a*	62 %	n/a*	67 %	72 %
Q4 AC coverage	23	25	25	25	24	25	25	25	23
Q5 lint warnings	2	1	1	1	0	4	3	2	2
Q6 typecheck errors	0	0	0	0	0	0	0	0	0
Q7 složitost	2	0	1	1	0	4	1	1	2
Q8 design quality	1/3	1/3	3/3	2/3	2/3	2/3	2/3	2/3	2/3
E1 in/out/cache (tis.)	115/60/11k	76/41/3k	62/30/5k	81/36/6k	65/23/3k	72/32/5k	92/36/8k	78/40/6k	55/23/4k
E2 trvání (min)	33	37	25	26	13	39	24	28	21
E3 počet kompakcí	0	0	0	0	0	0	0	0	0

[‡] P7 v r5 hodnotila specifikační issue, ne agentovu.

* Stryker nedokončil analýzu, protože agentovy testy selhávají a mutace nelze odlišit od výchozího selhání testů (r4, r5, A-2).

Barvy v procesních a produktových metrikách odpovídají exit kritériím z tabulky 10. Zelená znamená splnění, červená nesplnění a šedá neměřitelnou nebo deskriptivní hodnotu. Q4 v uložených judge artefaktech historicky používalo 24bodové hodnoticí schéma. Ve thesis jsou hodnoty převedeny na 25 AC po manuálním dopočítání AC25 (*custom holiday calendar*).

5. Vyhodnocení a diskuse

Výsledky případové studie ukazují, že samotná binární úspěšnost nepostačuje k posouzení chování AI coding agenta. Stejný běh může vytvořit téměř funkční řešení, ale současně porušit očekávaný pracovní postup nebo zanechat hůře udržovatelný kód. Interpretace se proto zaměřuje na to, co jednotlivé skupiny metrik zachytily, jak pomohly řídit úpravy instrukcí a kde naopak narážejí limity případové studie.

5.1 Interpretace výsledků

Tři cíle práce se hodnotí podle toho, co případová studie umožnila pozorovat. Interpretace proto postupuje od sady metrik přes iterativní úpravy instrukcí k ablacím.

5.1.1 Cíl 1: Sada metrik

Na základě analýzy existujících standardů kvality softwaru a současných benchmarků pro AI agenty navrhnout sadu metrik pokrývajících proces, kvalitu produktu a zdroje (dimenze, které stávající benchmarky neměří).

Sada 19 metrik byla navržena (kapitola 3) a použita na devíti bězích (kapitola 4). Její přínos nespočívá v tom, že „ukázala nějaká čísla“, ale v tom, že rozšířila hodnocení agenta za hranici binární úspěšnosti. Právě to bylo potřeba pro problém vymezený v kapitole 1. Umožňuje odlišit řešení, které jen projde testy, od řešení, které je zároveň vytvořeno obhajitelným postupem a zanechává použitelný kód.

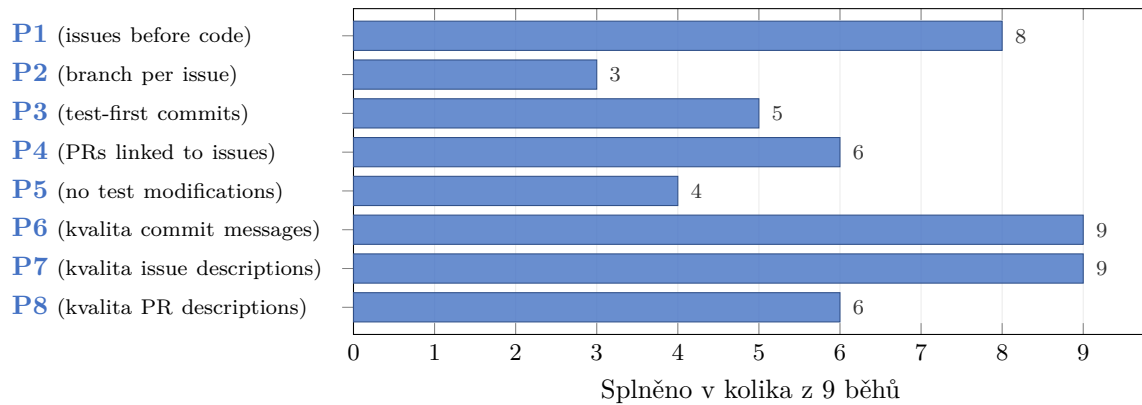
Procesní metriky (P1–P8)

procesní metriky (P1–P8) měří zda agent dodržoval vývojové praktiky (issues, branching, test-first, PR), tedy dimenzi kterou stávající benchmarky pokrývají jen okrajově (sekce 2.2.3). Na devíti bězích se ukázaly jako nejcitlivější složka sady. Byly jediné metriky, které kolísaly i mezi běhy se stejnými instrukcemi. Možným vysvětlením je, že kvalita produktu je u tohoto modelu řízena převážně tréninkovými daty (Q metriky zůstaly stabilní), zatímco dodržování vícekrokového postupu (issue → branch → test → implement → PR) je náchylnější k selhání, protože agent musí udržet sekvenci kroků v kontextu a každý krok je příležitost k odchylce. P1–P5 ale říkají jen splněno/nesplněno, což přináší ztrátu nuancí. Například P3 (test-first commits) hlásila v prvních dvou bězích stejný výsledek “nesplněno”, ale z git historie a behaviorálního popisu bylo vidět, že se chování lišilo. V prvním běhu agent testy nepsal vůbec, v druhém je psal ale kombinoval s implementací do jednoho commitu. Odpověď ano/ne řekne *co* nefunguje,

ale ne *proč*. K diagnostice a návrhu opravy instrukcí bylo vždy nutné doplnit binární metriky o podrobnější analýzu chování agenta (git historie, `FINDINGS.md`).

Podobně P4 měří *zda* agent vytvořil PR linkovaný na issue, ale neposuzuje kvalitu samotného popisu PR. Tuto mezeru doplňují judge-based metriky P6–P8, které hodnotí kvalitu commit zpráv, issue popisů a PR popisů na škále 1–3. V běhu r1 P4 hlásila “splněno” (agent skutečně vytvořil linkované PR), ale P8 = 2/3 zachytila, že část PR slučovala více issues do jednoho review artefaktu. Binární a judge-based metriky tedy měří odlišné aspekty téhož procesu a jejich kombinace se ukázala jako nutná.

Obrázek 9 shrnuje míru splnění exit kritérií procesních metrik napříč devíti běhy. Nejméně často byly splněny P2 (3/9) a P5 (4/9), zatímco P6 a P7 splnily exit kritérium ve všech bžích. Deterministické procesní metriky tak vykazují vyšší variabilitu než judge-based metriky procesních výstupů.



Obrázek 9: Míra splnění exit kritérií procesní metriky (P1–P8) napříč devíti běhy.

Kvalita produktu (Q1–Q8)

Na rozdíl od procesních metrik vykazují produktové metriky (Q1–Q8) celkově vyšší stabilitu. Q1 (API contract match) a Q6 (typecheck errors) byly splněny ve všech devíti bžích a Q4 (AC coverage) dosáhla ve většině hodnocených běhů plného skóre 25/25. Data naznačují, že základní kvalita produktu je u tohoto modelu řízena převážně tréninkovými daty, zatímco procesní chování závisí na instrukcích výrazněji. Uvnitř skupiny se ale metriky chovají odlišně podle toho, co měří.

Korektnost (Q1, Q2). Q1 zůstala stabilní ve všech bžích. Agent vždy vytvořil implementaci odpovídající API kontraktu, takže Q1 na této úloze nepřinesla rozlišení. Q2 (úspěšnost referenčních testů) je naopak nejvariabilnější produktová metrika. Exit kritérium 42/42 nesplnil žádný autonomní běh. Nejblíže byl pilotní běh r3 s hodnotou 41/42. Opakovaně selhávaly testy pokrývající přechody pause/resume. Specifikace i API kontrakt jednoznačně vyžadují zachování elapsed time (pole `pausedElapsed`), ale agent toto pole opakovaně ignoroval. Kontrast mezi Q1 (binární, vždy splněna) a Q2 (poměr projitých testů, variabilní) ukazuje,

že binární metrika nezachytí problémy které jemnější měření odhalí.

Testování (Q2–Q4). Tři metriky pokrývají testování z různých úhlů: Q2 (úspěšnost referenčních testů) měří kolik referenčních testů projde (vyšší = lepší implementace), Q3 (mutation score) měří jaký podíl uměle zavedených chyb agentovy testy odhalí (vyšší = kvalitnější testy) a Q4 (AC coverage) měří kolik acceptance criteria má odpovídající test (vyšší = úplnější pokrytí).

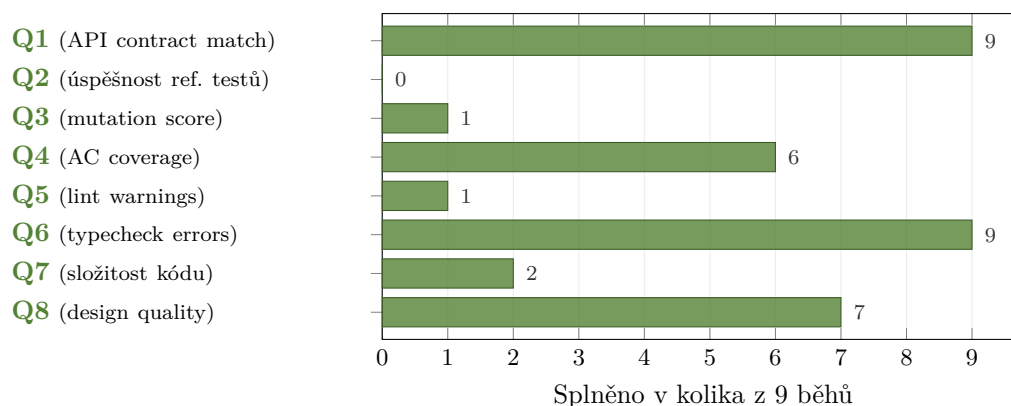
Vztah mezi Q2 a Q3 se ukázal jako nejinformativnější. V ablačním běhu B-2 agent dosáhl $Q2 = 11/42$. Jeho implementace byla z velké části chybná. Přesto $Q3 = 72\%$, tedy testy které napsal by v korektní implementaci zachytily většinu uměle zavedených chyb. Agent tedy uměl psát kvalitní testy, ale neuměl správně implementovat logiku. Kdyby sada obsahovala jen jednu z těchto metrik, tento rozdíl by zůstal skrytý.

Q4 zůstala relativně stabilní. Po manuálním doplnění AC25 dosáhla většina hodnocených běhů plného skóre 25/25. Tato metrika ale měří pouze *přítomnost* testu na dané AC, ne jeho správnost. Agent mohl napsat test který judge vyhodnotí jako pokrytí, ale který ve skutečnosti chování neověřuje. Kvalitu testů měří až Q3. Společně tyto tři metriky tvoří hierarchii: Q4 říká *co* agent testuje, Q3 *jak dobře*, a Q2 *zda* implementace skutečně funguje.

Čistota a design (Q5–Q8). Q5 (lint warnings) a Q7 (složitost kódu) reagovaly na změny v instrukcích nesterjně podle toho, zda obsahovaly verifikační kroky. Podrobnější rozbor přináší ablační analýza (sekce 5.1.3). Q6 (typecheck errors) naproti tomu zůstala nulová ve všech bžích. Statické typování poskytuje modelu explicitní vodítka přímo v kódu. Typy parametrů, návratové hodnoty a rozhraní jsou součástí trénovacích dat a model je při generování dodržuje. U dynamicky typovaných jazyků by Q6 pravděpodobně rozlišovala lépe.

Q8 (design quality) doplňuje deterministické metriky o strukturální pohled na kód, který statické nástroje v této sadě nezachycují. V pilotních bžích Q8 sledovala vývoj dokumentace napříč iteracemi. R1 a r2 dostaly 1/3 (žádné JSDoc, pod exit kritériem), r4 a r5 dostaly 2/3 (formálně splňuje exit kritérium, ale dokumentace jen na hlavním API, ne na utility funkcích). Plnou dokumentaci (3/3) přinesla teprve r3 po přidání explicitního verifikačního kroku do instrukcí. V ablačních bžích bez sekce Package Quality Q8 navíc zachytila ztrátu dokumentace na pomocných funkcích v obou bžích a v B-2 i rozpad modulární struktury do jednosouborového monolitu. Deterministické metriky Q5–Q7 ani jeden z těchto problémů nehlásily. Kombinace deterministické metriky a judge-based metriky tak zachycuje víc než kterýkoliv typ sám.

Obrázek 10 shrnuje míru splnění exit kritérií produktových metrik napříč devíti běhy. Q1 a Q6 byly splněny ve všech bžích, zatímco Q2 nesplnil žádný autonomní běh. Q3 splnil jeden ze šesti měřitelných běhů; u tří běhů (r4, r5, A-2) nebylo možné mutation score vyhodnotit.



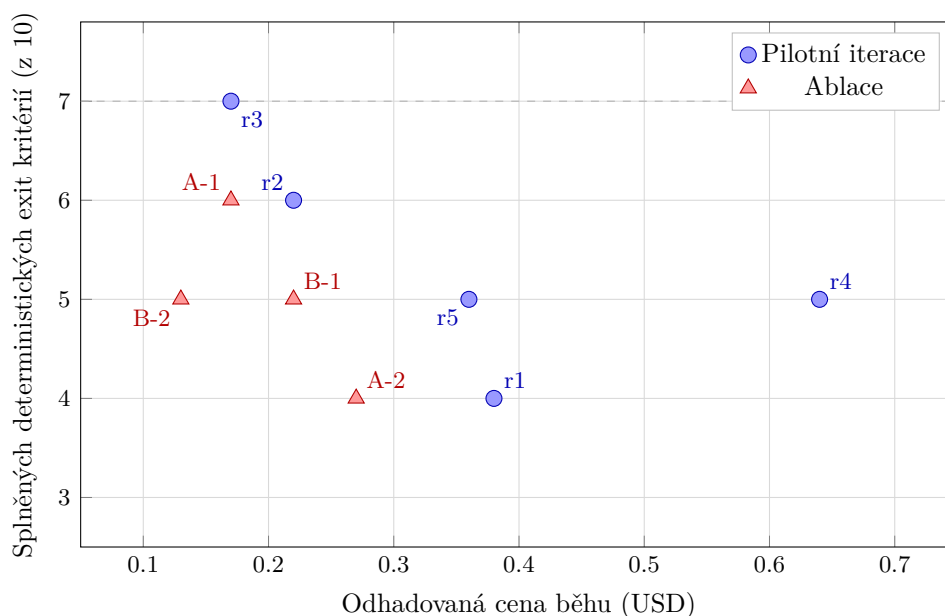
Obrázek 10: Míra splnění exit kritérií produktové metriky (Q1–Q8) napříč devíti běhy.

Zdroje (E1–E3)

E1 (vstup/výstup/cache tokeny) ukázala užitečný vzorec. Bez verifikačních kroků agent spotřeboval přibližně dvakrát více testovacích cyklů, což se projevilo na vyšší spotřebě tokenů. E2 (trvání) závisí převážně na rychlosti poskytovatele API (rate limity, latence), ne na kvalitě agentovy práce. Nejkratší běh (r5, 13 min) vznikl tím, že agent ignoroval celý pracovní postup. E2 je proto nutné číst společně s procesními metrikami.

E3 (počet kompakcí) ve všech devíti bězích = 0 potvrzuje, že kontext zůstal celistvý. Žádný běh nevyčerpал kontextové okno a prostředí nemuselo historii zhušťovat. E3 je proto deskriptivní indikátor stability kontextu během běhu, ne exit kritérium. Na této úloze plní roli kontextového faktu, že naměřené hodnoty ostatních metrik nebyly zkresleny ztrátou detailu z kompakce. Pro rozsáhlejší projekty s delšími sessiony by E3 rozlišovala výrazněji a stejný kontextový fakt by pak mohl naopak indikovat tlak na kontext.

Při orientačním přepočtu na cenu (ceník poskytovatele Alibaba Cloud Model Studio) se ukázalo, že vyšší spotřeba tokenů nekoreluje s lepším výsledkem. Nejlevnější běh (r3, přibližně \$0,17) dosáhl nejlepších metrik (Q2 = 41/42 a plné splnění P1–P5), zatímco nejdražší běh (r4, přibližně \$0,64) měl horší výsledky. Agent v r4 spotřeboval víc tokenů na cyklické debugování, ne na kvalitnější práci. Obrázek 11 ukazuje vztah mezi odhadovanou cenou běhu a počtem splněných deterministických exit kritérií. Běh r3 dosáhl nejvyšší hodnoty 7/10 při nejnižší ceně z pilotních iterací. Ablacní běhy se pohybují v rozmezí 4–6 splněných kritérií.



Obrázek 11: Vztah mezi odhadovanou cenou běhu a počtem splněných deterministických exit kritérií.

Shrnutí a reflexe

Sada se ukázala jako proveditelná pro opakované měření. deterministické metriky se extrahují automatizovaně a judge-based metriky hodnotí LLM-as-judge s fixním hodnoticím schématem. Jedno kompletní měření trvá řádově minuty, což umožňuje zpětnou vazbu v rámci jedné iterace.

Kritický pohled na design sady odhaluje několik slabých míst. Dvě binární exit kritéria (Q1, Q6) byla splněna ve všech devíti bězích, takže pro rozlišování běhů na této úloze nepřinesla informaci. Jsou pojistkou pro jiné úlohy nebo jazyky. Deskriptivní E3 zůstala konstantně 0. Žádný běh tedy neprošel kompakcí kontextu, takže rozdíly mezi běhy nelze vysvětlit ztrátou části historie agenta. Procesní metriky přinesly nejvíc nových poznatků (nestabilita chování kterou produktové metriky nezachytily), ale jsou zároveň nejméně spolehlivé. Kolísají i mezi běhy se stejnými instrukcemi, takže u jednoho běhu jim nelze plně věřit. Binární P1–P5 navíc ztrácí nuance, protože “nesplněno” neříká *proč* a k diagnostice bylo vždy nutné vrátit se k transcriptu agenta. Kvalitativní metriky (Q4, Q8) zachytily problémy které deterministické nevidí, ale bez validace Cohenovým κ zůstávají podpůrnými indikátory.

5.1.2 Cíl 2: Iterativní postup

Na případové studii ověřit proveditelnost iterativního postupu návrhu instrukcí řízeného těmito metrikami a vyhodnotit, zda a jak vede k měřitelným změnám v chování agenta podle navržených metrik.

ale tento problém neodstranily. To ukazuje hranici instrukcí. Pokud agent zvolí intuitivní, ale nesprávný implementační vzorec, samotné pravidlo ani kontrolní krok nemusí stačit. Podobně nelze vyloučit, že některé regrese v r4 byly vedlejším efektem úprav cílených na jiné metriky. Nedeterminismus modelu a malý počet běhů neumožňují odlišit kauzální efekt úpravy od náhodného šumu.

Z hlediska praktické proveditelnosti běh agenta v jednotlivých iteracích trval 13–37 minut (medián 26 minut) plus řádově desítky minut na diagnostiku a úpravu instrukcí. Diagnóza se opírala o tabulku metrik, behaviorální popis z `FINDINGS.md` a analýzu transcriptu agenta. Zjištěný vzorec na spektru operacionalizace (pravidlo → příkaz → verifikační krok) považujeme za jeden z hlavních poznatků práce, ale vychází z jedné případové studie s jedním modelem a z domény, kde existují deterministické nástroje zpětné vazby. Jeho platnost pro jiné modely, typy úloh a domény mimo implementaci kódu ze specifikace vyžaduje další výzkum.

5.1.3 Cíl 3: Ablace

Z instrukcí vytvořených v cíli 2 prozkoumat ablacemi, které složky přispívají k měřenému chování agenta a které jsou redundantní.

Data z ablace A jsou konzistentní s tím, že verifikační kroky v tomto prostředí pomáhaly. Po jejich odebrání agent verifikační příkazy nespouštěl spolehlivě a klesly vybrané metriky kvality i korektnosti. Verifikační kroky se proto v této případové studii jeví jako prakticky důležitá složka pracovního postupu. Vzhledem ke dvěma běhům na variantu a výrazné variabilitě mezi běhy však nejde o statistický důkaz kauzálního efektu.

Data z ablace B jsou konzistentní s tím, že sekce Package Quality byla v deterministických metrikách převážně redundantní. Po jejím odebrání zůstaly Q5, Q6 a Q7 na srovnatelné úrovni. Změnu ve strukturální kvalitě kódu zachytila hlavně Q8. To naznačuje, že kódové konvence nemusí působit jako přímé vynucení, ale mohou ovlivňovat rozhodnutí, která deterministické nástroje v této práci nezachycují.

Variabilita procesních metrik byla v ablacích srovnatelná s pilotními iteracemi, přestože instrukce pro pracovní postup zůstaly beze změny. Data naznačují, že dodržení procesních pravidel je u tohoto modelu stochastická vlastnost, kterou instrukce ovlivňují jen částečně. Variabilita mezi B-1 ($Q2 = 37/42$) a B-2 ($Q2 = 11/42$) ukazuje, že nedeterminismus modelu může být silnějším faktorem než efekt ablace. Při dvou bězích na každou variantu nelze oba faktory spolehlivě odlišit.

Reflexe designu ablací

Vzorec operacionalizace (pravidlo → příkaz → verifikační krok), identifikovaný v cíli 2, jsme pojmenovali až při analýze pilotních dat. Výběr ablací mu předcházel. Zpětně je zřejmé, že u ablace A byl pokles metrik do značné míry předvídatelný. `npx eslint` a `git log`

jsme do instrukcí přidali v pilotních iteracích právě proto, že bez nich metriky selhávaly. Novým poznatkem byla dvojí funkce verifikačních kroků. Kromě kontroly kvality strukturují práci agenta a zabráňují cyklickému debugování (sekce 4.3.2). Informativnější by byla ablace Constraints nebo struktury pracovního postupu, kde výsledek z pilotů předvídatelný není. To zůstává námětem pro další výzkum.

5.2 Porovnání s literaturou

5.2.1 Srovnání sady metrik

Případová studie konkretizuje problém vymezený v kapitole 2. Literatura ukazuje, že funkční úspěšnost sama neříká, zda by patch obstál v code review, zda je výsledný kód udržitelný a zda agent dodržel očekávaný pracovní postup. Stejně rozlišení se ukázalo i v této studii. Běhy s podobnou funkční úspěšností se lišily v tom, zda agent dodržel pracovní postup, zanechal auditovatelnou stopu a vytvořil udržitelný kód.

Funkční korektnost proto zůstává nutnou složkou hodnocení, ale sama nestačí. Přidaná hodnota navržené sady metrik je v tom, že odděluje výsledek, proces a zdrojovou náročnost.

Odlišnost vůči existujícím benchmarkům je i v tom, že metriky zde nejsou jen evaluačním výstupem, ale i řídicím nástrojem pro další iteraci. To je důležité pro hlavní problém práce. Nestačí vědět, že agent selhal nebo uspěl, pokud z výsledku neplyne, jak instrukce upravit. V tomto smyslu je navržená sada metrik bližší praktickému diagnostickému nástroji než klasickému benchmarku.

5.2.2 Vliv instrukcí na chování agenta

Studie o instrukčních souborech zatím většinou měří jejich přítomnost nebo absenci jako celek. Lulla et al. (2026) spojují `AGENTS.md` s nižším mediánovým trváním běhu, zatímco Gloaguen et al. (2026) ukazují jen marginální přínos generických repository-level souborů a upozorňují na vyšší náklady. SkillsBench (X. Li et al., 2026) dále rozlišuje mezi kurátorovanou a automaticky generovanou instrukční podporou a ukazuje, že rozhodující je kvalita a relevance obsahu. Naše výsledky jsou s těmito zjištěními kompatibilní. Ne všechny složky `AGENTS.md` přispívají stejně.

Nejvýraznější rozdíl se v naší studii objevil u verifikačních kroků. Ablace A ukázala pokles jak ve funkční korektnosti, tak v deterministických metrikách kvality kódu, když byly explicitní kontroly odstraněny. To doplňuje pozorování, že opakování ignorované instrukce nepomáhá a účinnější je její přestrukturování do pracovního postupu (Breunig, 2025). Pilotní fáze v našich datech ukázala stejný vzorec. Obecné pravidlo nestačilo, konkrétní příkaz pomohl jen částečně a stabilnější zlepšení přinesl až verifikační krok. Tento vývoj je konzistentní i s výzkumem specifity instrukcí, podle něhož vyšší konkrétnost pomáhá zejména u procedurálních úloh, ale přílišný detail může omezovat flexibilitu modelu (Kim, 2025; Zi et al., 2025).

Sekce Package Quality se chovala odlišně. Její odebrání nezhoršilo deterministické metriky stejným způsobem jako odebrání verifikačních kroků, ale projevilo se v designové kvalitě hodnocené LLM-as-judge. Data jsou proto konzistentní s tím, že část instrukcí nefunguje primárně jako přímá kontrola chování, ale jako opora, která připomíná modelu relevantní znalosti z tréninku. V tomto ohledu je užitečná analogie s promptingem. Minimální nápověda může vyvolat žádoucí chování i bez detailního skriptu (Min et al., 2022; Wei et al., 2022). V této práci tento rozdíl interpretujeme jako rozdíl mezi *vynucením* a *aktivací*. Verifikační kroky vynucují měřitelné požadavky, zatímco obecnější konvence podporují designová rozhodnutí.

Zároveň ale platí, že vliv instrukcí nelze od nedeterminismu modelu oddělit dokonale. Razavi and Fard (2025) upozorňují na vysokou citlivost modelů na drobné změny promptu a naše pilotní i ablační běhy tento problém potvrzují. Proto je třeba naše výsledky číst jako indikativní syntézu na úrovni případové studie. Ukazují, které typy instrukcí se v tomto prostředí osvědčily, ale neprokazují univerzální kauzální zákon.

5.3 Omezení a jejich dopad

Sekce 3.5 identifikovala metodologická omezení designu studie předem. Zde ukazujeme, jak se tato omezení projevila v naměřených datech, a popisujeme omezení která se objevila až v průběhu studie. Omezení řadíme podle toho, co ohrožují. Nejprve sílu závěrů, pak spolehlivost měření a nakonec rozsah platnosti výsledků.

Nedeterminismus a citlivost na prompt. Pět binárních procesních metrik P1–P5 (splněno/nesplněno) kolísalo od 5/5 (r3) přes $\sim 3/5$ (r4) po $\sim 1/5$ (r5), přestože procesní instrukce v těchto bězích zůstaly stejné. R4 přidalo verifikační příkaz `npm vitest run` v Process a pravidlo o API kontraktu v Constraints (obojí cíleno na Q2). Dodržení procesních pravidel se přesto zhoršilo, přestože žádná z úprav na pracovní postup přímo necílila. Razavi and Fard (2025) tento jev popisují jako citlivost na prompt (*prompt sensitivity*). Alternativním vysvětlením je efekt kumulace instrukcí. S rostoucím počtem instrukcí klesá pravděpodobnost dodržení všech současně. Instrukce rostly z r1 do r5. Část poklesu procesní shody může být tímto efektem, ne jen citlivostí na formulaci.

Dva běhy na ablační variaci tento problém zmírňují, ale neposkytují statistickou sílu. Pokud oba běhy téže ablace ukazují stejný směr, je pravděpodobnější že jde o efekt ablace, ne o náhodu. Například Q5 (lint warnings) v ablaci A vykazovala v obou bězích 4 a 3 varování linteru, zatímco r3 (plné instrukce) hlásil 1. Shoda obou běhů zvyšuje důvěru, že odebrání sekce Process skutečně ovlivnilo Q5. Naopak u Q2 (úspěšnost referenčních testů) v ablaci B jeden běh dosáhl 37/42 projitých testů, druhý jen 11/42. Tak velký rozptyl mezi dvěma běhy se stejnými instrukcemi znamená, že rozdíl oproti r3 mohl být způsoben nedeterminismem modelu, ne ablací. Výsledky ablací proto interpretujeme jako indikace, ne jako statisticky podložené závěry. V kombinaci s vedlejšími efekty popsány výše (r4) to znamená, že každá úprava instrukcí vyžaduje měření celé sady metrik, ne jen těch na které úprava cílila.

Efekt učení autora. Zlepšení mezi iteracemi nemusí plynout jen z lepších instrukcí. Běh r1 splnil P1, P4, Q1 a Q6, zatímco r3 navíc splnil P2–P5. Autor se v průběhu iterací učil diagnostikovat selhání a formulovat opravy. Tento efekt nelze od efektu instrukcí oddělit. Diagnostická fáze (výběr příčiny selhání a návrh opravy) závisí výhradně na autorovi a nezávislé ověření nebylo provedeno. Částečnou mitigací je použití strukturovaného diagnostického rámce (sekce 3.1.1). Diagnóza se opírala o naměřená data a literaturou podložené principy, ne o intuici. Přesto nelze vyloučit, že zkušenější autor by ze stejné výchozí varianty dosáhl lepších výsledků rychleji.

Změny konfigurace mezi běhy. Běhy r1–r2 proběhly bez kontejnerové izolace, od r3 běží agent v Docker kontejneru. Od r4 byla změněna konfigurace modelu (parametry kontextového okna). Tyto změny jsou potenciálním zavádějícím faktorem. Regrese r4 mohla být částečně způsobena změnou konfigurace, ne pouze přidáním dvou řádků do Constraints. Hlavní proměnné (model, instrukce, specifikace) zůstaly konzistentní. Temperature modelu nebyla explicitně nastavena. Experiment používal výchozí konfiguraci poskytovatele.

Tiché ukončení nástroje. Při ablačních bězích se OpenCode v některých případech tiše ukončil uprostřed práce agenta. Běhy s vyšší spotřebou kontextového okna byly postiženy častěji (ablace A spustila více testovacích cyklů než ablace B, viz sekce 4.3.2). Postižené běhy byly opakovány. Pravděpodobnou příčinou je běh agenta uvnitř Docker kontejneru (spojení s API poskytovatele, limity paměti nebo síťové timeouty), ale příčina nebyla potvrzena.

Nerozlišující exit kritéria. Q1 (API contract match) a Q6 (typecheck errors) prošly ve všech devíti bězích, takže pro rozlišení běhů na této úloze nepřinesly informaci. Pravděpodobným vysvětlením je design úlohy. Specifikace poskytuje explicitní TypeScript typy, takže shoda s kontraktem (Q1) i typová kontrola (Q6) jsou pro model nízkonákladové. E3 je deskriptivní indikátor, ne exit kritérium, a ve všech bězích = 0. Pro tuto úlohu má roli kontextového faktu o stabilitě kontextu během běhu. Na rozsáhlejších projektech by mohla diskriminovat výrazněji.

Judge-based metriky bez validace. Hlavní závěry studie stojí na deterministické metrikou. Judge-based metriky slouží jako podpůrné indikátory, protože jejich spolehlivost nebyla validována proti lidskému hodnocení (Cohenovo κ). Jedinou z nich, na které stojí netriviální závěr, je Q8 (design quality). Jako jediná metrika zachytila pokles designové kvality v ablacích, který deterministické metriky nehlásily. Bez κ validace ale nelze sílu tohoto závěru plně posoudit.

Optimalizace na fixní sadu metrik. Iterativní postup ladí instrukce proti pevně dané sadě 19 metrik. S každou iterací roste riziko, že úprava cílí na metriku samotnou, ne na chování, které metrika měří. Příkladem je přidání `npx eslint` do verifikačních kroků. Q5 kleslo, ale agent kvůli tomu nepíše čistší kód, jen ho po sobě kontroluje. Goodhartův zákon je v úzké sadě metrik akutnější. Pokrytí třemi dimenzemi ho částečně omezuje, protože zlepšení jedné metriky bez zlepšení reálného chování se obvykle projeví zhoršením v jiné. Spolehlivá ochrana proti přeučení by ale vyžadovala validaci na nezávislé sadě metrik nebo na jiném projektu, což přesahuje rozsah této studie.

Kontrolované podmínky vs. reálný vývoj. Experiment probíhal v kontrolovaném prostředí, tedy v prázdném repozitáři, jedna úloha s kompletní specifikací, fixní sada metrik. Reálný vývoj zahrnuje nejednoznačné požadavky, měnící se specifikace a integraci s existujícím kódem. Instrukce optimalizované pro kontrolované podmínky (verifikační kroky, explicitní příkazy) nemusí fungovat v prostředí kde specifikace není kompletní nebo se mění v průběhu práce.

Jeden model, jeden nástroj, jeden projekt. Omezení plynoucí z jednoho modelu, jednoho projektu a deterministické specifikace diskutuje sekce 3.5. V praxi se tato omezení projevila zejména u procesní metriky (P1–P8). Variabilita neumožňuje odlišit efekt instrukcí od nedeterminismu modelu bez většího počtu běhů. Výsledky jsou navíc specifické pro OpenCode a model MiniMax-M2.5. Jiné nástroje (GitHub Copilot, Cursor, Claude Code) mohou zpracovávat soubory s instrukcemi odlišně. Zda by instrukce fungovaly stejně na projektech s vyšší nejednoznačností specifikace, na jiných modelech nebo s jinými nástroji, zůstává otevřenou otázkou (sekce 5.5).

5.4 Doporučení pro praxi

Následující doporučení vycházejí z pilotních iterací a ablací popsaných v kapitole 4. Konkrétní hranice kritérií platí pro tento model a typ projektu. Přenositelný je iterační postup (měř → diagnostikuj → uprav → měř znovu) a principy strukturování instrukcí.

Operacionalizace instrukcí

V pilotních iteracích se opakoval vzorec. Obecné pravidlo (“každé pole musí být použito”) agent ignoroval, zatímco explicitní příkaz s okamžitou zpětnou vazbou (`npx eslint`, `tsc -noEmit`) dodržoval spolehlivě i v bžích, kde ignoroval zbytek pracovního postupu (sekce 4.3.2). Ablací běhy ukázaly komplementární stranu. Odstranění verifikačních kroků (ablace A) vedlo k poklesu Q2 (úspěšnost referenčních testů) z 41/42 na 35–37/42 a nárůstu Q5 (lint warnings) z 1 na 3–4 varování. Breunig (2025) formuloval stejný princip, že když agent instrukci ignoruje, nepomůže ji zopakovat. Je třeba ji převést na konkrétní akci. Spektrum operacionalizace (obrázek 12) popisuje, po jaké ose se intervence pohybují. Čím blíže je instrukce verifikovatelnému příkazu, tím spolehlivěji ji agent v této případové studii dodržoval.

Ablace B ale ukázala i druhou stránku. Odstranění sekce Package Quality ponechalo deterministické metriky (deterministické metriky) beze změny, zatímco Q8 (design quality) kleslo z 3/3 na 2/3 (sekce 4.3.3). Konvence zřejmě nepůsobí jako přímé vynucení, ale spíš jako *aktivace* latentních znalostí modelu, tedy jako připomínka toho, co se model v tréninkových datech naučil. Toto rozlišení je interpretační pojmenování výsledků této studie, ne převzatá taxonomie z literatury. Z toho plyne praktické rozlišení. Měřitelné požadavky je vhodné převést na verifikovatelné kroky ve verifikační části před PR, zatímco požadavky, které nelze měřit deterministicky (srozumitelnost dokumentace, vhodnost dekompozice), strukturovat

jako nápoředu, která modelu řipomíná řádoucí řměř. Úřinnost deterministické řvětvě řitřom řávisí na existenci nástroje s jednoznačným výřstupem. Tam, kde takový nástroj řhybí, je řmíra operacionalizace principiálně nižřší (sekce 5.5). Závěr o designové kvalitě stojí na Q8, tedy na podpůřrné judge-based metrice bez κ validace, a je proto nutné jej říst opatřně.

Počítat s nedeterminismem a měřit proces

Agent dodřřžoval verifikační řpřikazy deterministicky, ale řvěcekrokový pracovní řpostup (issue $\rightarrow$ branch $\rightarrow$ test $\rightarrow$ PR) nedeterministicky. řpilotní řběh r3 dosáhl P1 (issues before code) = 5/5, řběh r5 řpři řtěměř identických řinstrukcích P1 = $\sim 1/5$ (sekce 5.3). Na úrovni výřsledku (Q2) řpitřom řrozřdíl nemusel řbýt patřrný. procesní metriky (P1–P8) zachyřtily řproblém, který produktové metriky (Q1–Q8) samy o sobě neodřhalily. Doporučením je nespřolěhat na řjednorázové ověření výřřstupu, řzavést měřitelná řexit řkritéria pokrývající proces i řkvalitu a řkontřolovat je opakovaneř.

Iterovat na řzákladě dat

Řpřínos této řpráce nespočívá v konkréřtních řinstrukcích, ale v řpostupu jejich řnávrhu. Je řtřeba naměřit metriky, řdiagnostikovat řpřičinu řselhání, řprovést řcílenou úřpravu řinstrukcí a řzměřit znovu (sekce 3.1.1). Tento řcyklus lze řpřenést na řjiný model nebo řprojekt řjen řpo řpřizpůsobení řmetrik a řexit řkritérií řdanému řprostředí. Řpodmínkou je řsada řmetrik pokrývající řvíce řdimenzí (proces, řkvalitu řproduktu, řzdroje), řprotože řjednodimenzionální měření (např. řjen test pass rate) řmůže řzakřřýt řproblém v řjiné řdimenzi (sekce 5.1).

Řpraktický řdůřsledek je řpřímočarý. Řklíčové řpořřadavky je řúčelněřřší řpřevádět do měřitelného řpracovního řpostupu s řjasnými řverifikačními řkroky řnež je řjen řpřidávat jako řdalřší řobecná řpravidla.

5.5 Náměty pro řdalřší výřřzkum

Řpřířpadová řstudie ořtevřřela řněkolik řotázek, které řpřesahují řrozsah této řpráce.

Nedeterminismus řvěcekrokového řpracovního řpostupu

Agent dodřřžoval řverifikační řpříkazy (řjednokrokové řakce) deterministicky, ale řvěcekrokový pracovní řpostup (issue $\rightarrow$ branch $\rightarrow$ test $\rightarrow$ implement $\rightarrow$ PR) nedeterministicky. R3 řkompleřně, řr5 řvřubec, řpři řtěměř identických řinstrukcích. Řmořžná řvysvětlení řzahrnují řztrátu řpozornosti v řdlouhém řkontextovém řokně, řstochastickou řvolbu řmezi řstrukturovaným a řnestrukturovaným řvýřvojem z řtréninku řmodelu a řpozici řinstrukce v řkontextu. Řgranulární řablace řpracovního

postupu (které kroky jsou kritické?) a opakované běhy se stejnými instrukcemi by tento fenomén pomohly lépe pochopit.

Kontinuita práce mezi agenty

procesní metriky (P1–P8) měří transparentnost artefaktů (issues, zprávy commitů a popisy PR). Otevřenou otázkou je, zda tato transparentnost k tomu, aby jiný agent (nebo člověk) navázal na rozpracovaný projekt? Testování handoff scénářů by ověřilo, zda procesní instrukce vytváří nejen pozorovatelný, ale i přenositelný pracovní kontext.

Automatizace iteračního postupu

Fáze Diagnóza a Úprava jsou v této práci prováděny manuálně autorem. Frameworky jako PromptWizard (Agarwal et al., 2024) a Prompt Alchemy (Ye et al., 2025) ukazují, že cyklus analýzy selhání a návrhu úprav lze automatizovat. Kombinace automatizovaného prompt optimizery s metrikami P/Q/E by umožnila rychlejší iteraci bez manuální diagnózy.

Ověření načtení instrukcí

Pilotní data ukazují, že agent instrukce přečte (v transcriptu cituje jejich obsah), ale ne vždy je aktivuje jako řídicí smyčku běhu. Místo pracovního postupu z `AGENTS.md` použije vlastní postup odvozený ze specifikace. Ověřitelný krok na začátku běhu (agent musí explicitně potvrdit přečtení a aktivaci pracovního postupu před prvním zápisem kódu) by mohl tento problém adresovat.

Mechanismy účinku instrukcí

Naše data naznačují dva odlišné mechanismy, které v této práci označujeme jako *vynucení* (verifikační kroky přímo kontrolují výstup) a *aktivace* (obecné konvence připomínají modelu znalosti z tréninku). Analogie existuje v prompt engineeringu. Chain-of-thought prompting ukazuje, že minimální náповěda může aktivovat komplexní latentní schopnost modelu efektivněji než detailní instrukce. Systematický experiment s odstupňovanou specificitou instrukcí, od náповěd (“piš modulární kód”) přes pravidla (“každé pole musí být použito”) po verifikační kroky (“spusť `tsc`, oprav chyby”), by mohl identifikovat, která míra konkrétnosti je pro který typ požadavku optimální. Otevřená je i otázka, zda se toto optimum posouvá se schopnostmi modelu. Lepší modely by mohly potřebovat méně specifické instrukce (analogicky k expertise reversal effect v kognitivní psychologii, kde instrukce účinné pro nováčky zpomalují experty).

Generalizovatelnost verifikačních kroků

Účinnost verifikačních kroků v naší studii závisela na existenci deterministických nástrojů (`eslint`, `tsc`, `test runner`), které produkují jednoznačný výstup zpět do kontextu agenta. Kódování je v tomto ohledu specifická doména. Řada požadavků na kvalitu má odpovídající nástroj. U jiných typů úloh (dokumentace, architektonická rozhodnutí, plánování) takové nástroje často neexistují a operacionalizace se může zastavit na úrovni pravidla nebo příkazu bez externí zpětné vazby. Otevřená otázka je, zda principy operacionalizace platí i mimo kódování, nebo zda je jejich účinnost podmíněna dostupností deterministické zpětné vazby.

Rozšíření na další modely, projekty a domény

Výsledky platí pro jeden model (MiniMax-M2.5) a jeden typ projektu (deterministická business logika). První osa dalšího výzkumu je replikace uvnitř programování. Je potřeba ověřit jiné modely, jiné agentní nástroje a projekty s vyšší nejednoznačností specifikace (sekce 5.3). Druhá osa je ověření mimo programování, kde nemusí existovat ekvivalent `test runneru`, `lintu` nebo typové kontroly. Právě tam se ukáže, zda vzorec pravidlo $\rightarrow$ příkaz $\rightarrow$ verifikační krok vyjadřuje obecnější princip řízení agentů, nebo hlavně výhodu programovací domény s deterministickou zpětnou vazbou.

Exit kritéria jako součást instrukcí

V naší studii zůstávala exit kritéria mimo `AGENTS.md`. Agent dostal instrukce, *jak* pracovat, ne *jakých hodnot* metrik dosáhnout. Otevřenou otázkou je, zda by zahrnutí kritérií přímo do instrukcí, například jako pre-PR self-check (“spust `tsc --noEmit`, žádné chyby”), výsledky zlepšilo, nebo zda by agent kritéria začal optimalizovat triviálními zkratkami (verbose kód proti prahu složitosti, povrchní testy proti pokrytí). Riziko Goodhartova zákona je v úzké sadě metrik vyšší. Sada pokrývající tři dimenze ho částečně omezuje, protože gaming jedné metriky se projevuje v jiné. Hranice mezi legitimním řízením a změnou povahy měřeného chování zůstává otevřenou otázkou.

Závěr

Tato práce navrhla sadu metrik a iterativní postup pro systematické navrhování a vyhodnocování instrukcí pro AI coding agenty a ablacemi prozkoumala vliv jednotlivých složek instrukcí. Pro všechny tři cíle případová studie systému upomínek faktur ověřila proveditelnost navrženého postupu na jednom případě a přinesla indikativní poznatky. Studie zahrnovala pět pilotních iterací a dvě ablace, každou ověřenou dvěma nezávislými běhy.

Prvním cílem bylo navrhnout sadu metrik pokrývajících proces, kvalitu produktu a zdroje, a tím rozšířit hodnocení o procesní, produktovou a zdrojovou dimenzi. Výsledná sada 19 metrik pokrývá tři dimenze odvozené z taxonomie Fentona a Biemana. Procesní metriky odhalily problémy, které výsledkové metriky samy o sobě nezachytily. Běh s téměř plným skóre funkčních testů vykazoval nestabilní dodržení procesních pravidel. Kombinace deterministických a judge-based metrik přinesla komplementární pohled, protože každý typ zachycuje jinou dimenzi chování agenta.

Druhým cílem bylo na případové studii ověřit proveditelnost iterativního postupu návrhu instrukcí řízeného těmito metrikami a vyhodnotit, zda a jak vede k měřitelným změnám v chování agenta. Metriky zachytily měřitelné změny mezi běhy, včetně zlepšení i regresí. Jedna iterace způsobila regresí, po které bylo nutné vrátit se k předchozí verzi instrukcí. Postup nevedl ke splnění všech exit kritérií. Zejména referenční testy zůstaly u nejlepšího běhu na hodnotě 41/42. Právě toto nesplnění ale metriky lokalizovaly jako stabilní slabinu, kterou další úpravy instrukcí neodstranily. Opakujícím se vzorcem úspěšných úprav bylo nahrazení obecného pravidla konkrétním příkazem s verifikačním krokem. Účinnost tohoto postupu závisela na dostupnosti deterministických nástrojů s jednoznačným výstupem. U požadavků bez takového nástroje zůstává instrukce na úrovni pravidla nebo příkazu bez vnější zpětné vazby.

Třetím cílem bylo ablacemi prozkoumat, které složky instrukcí vytvořených v cíli 2 přispívají k měřenému chování agenta a které jsou redundantní. Výsledky ablací byly konzistentní s interpretací, že klíčové metriky v této případové studii závisely na verifikačních krocích, což naznačila už pilotní fáze. Jejich odebrání zhoršilo funkční korektnost i čistotu kódu. Konvence kvality kódu automatizované metriky neovlivnily, ale designová kvalita hodnocená LLM-as-judge klesla. Výsledky zároveň ukázaly, že dodržení procesních pravidel je výrazně citlivé na nedeterminismus modelu. Stejná sada instrukcí proto může vést k odlišnému dodržení vícekrokového pracovního postupu.

Hlavním přínosem práce je iterativní postup a sada metrik, které může praktik aplikovat na vlastním projektu s vlastními exit kritérii. Konkrétní instrukce a naměřené hodnoty platí pro tuto případovou studii. Přenositelný je postup, ne výsledná čísla. Práce tím ověřuje proveditelnost navržené sady metrik a iterativního postupu na jednom případě, zatímco závěry mají povahu analytické generalizace. Formulují principy k dalšímu ověření, zejména otázku platnosti vzorce operacionalizace mimo programování, nikoliv statistické tvrzení o populaci.

Pro praktika z toho plyne jednoduché doporučení. Nestačí sledovat, zda agent úkol vyřešil, ale i jak k řešení došel a jak kvalitní kód zanechal. Pro akademického čtenáře práce ukazuje, že instrukce lze studovat ne jen jako promptový artefakt, ale jako samostatně měřenou nezávislou proměnnou, jejíž účinek je třeba hodnotit přes proces, produkt i náklady.

Literatura

- Agarwal, E., Dani, V., Ganu, T., & Nambi, A. (2024). PromptWizard: Task-Aware Prompt Optimization Framework. *arXiv preprint arXiv:2405.18369*. <https://arxiv.org/abs/2405.18369>
- Ammann, P., & Offutt, J. (2016). *Introduction to Software Testing* (2nd ed.). Cambridge University Press.
- Bacchelli, A., & Bird, C. (2013). Expectations, Outcomes, and Challenges of Modern Code Review. *Proceedings of the 35th International Conference on Software Engineering*, 712–721. <https://doi.org/10.1109/ICSE.2013.6606617>
- Beck, K. (2000). *Extreme Programming Explained: Embrace Change*. Addison-Wesley.
- Beller, M., Bholanath, R., McIntosh, S., & Zaidman, A. (2016). Analyzing the State of Static Analysis: A Large-Scale Evaluation in Open Source Software. *IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, 470–481. <https://doi.org/10.1109/SANER.2016.105>
- Beller, M., Gousios, G., Panichella, A., Proksch, S., Amann, S., & Zaidman, A. (2019). Developer Testing in the IDE: Patterns, Beliefs, and Behavior [Large-scale field study (2,443 engineers, 2.5 years, four IDEs) of how developers actually test. Findings: half do not test, TDD rarely practiced, test effort much lower than self-reported. Supports test-related practices as observable behavior distinct from stated ideals.]. *IEEE Transactions on Software Engineering*, 45(3), 261–284. <https://doi.org/10.1109/TSE.2017.2776152>
- Bissyandé, T. F., Lo, D., Jiang, L., Réveillère, L., Klein, J., & Le Traon, Y. (2013). Got Issues? Who Cares About It? A Large Scale Investigation of Issue Trackers from GitHub. *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*, 188–197. <https://doi.org/10.1109/ISSRE.2013.6698918>
- Boehm, B. W. (1981). *Software Engineering Economics* [Seminal work on software cost estimation; introduced the COCOMO (Constructive Cost Model) based on project size (SLOC) and effort multipliers.]. Prentice-Hall.
- Boehm, B. W., Abts, C., Brown, A. W., Chulani, S., Clark, B. K., Horowitz, E., Madachy, R., Reifer, D. J., & Steece, B. (2000). *Software Cost Estimation with COCOMO II*. Prentice Hall.
- Breunig, D. (2025). Don't Fight the Weights [Defines fighting-the-weights: when prompt instructions oppose trained model behavior; recognition signs and mitigation strategies]. <https://www.dbreunig.com/2025/11/11/don-t-fight-the-weights.html>
- Brooks, F. P. (1987). No Silver Bullet: Essence and Accidents of Software Engineering. *Computer*, 20(4), 10–19. <https://doi.org/10.1109/MC.1987.1663532>
- Ehsani, R., et al. (2026). Where Do AI Coding Agents Fail? An Empirical Study of Failed Agentic Pull Requests in GitHub [33k agent PRs, 5 coding agents, taxonomy of failures]. *Proceedings of MSR 2026*. <https://arxiv.org/abs/2601.15195>
- Fenton, N. E., & Bieman, J. (2014). *Software Metrics: A Rigorous and Practical Approach* (3rd ed.) [Process vs. product vs. resource metrics taxonomy; GQM framework]. CRC Press.

- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.
- Forsgren, N., Storey, M.-A., Maddila, C., Zimmermann, T., Houck, B., & Butler, J. (2021). The SPACE of Developer Productivity: There’s More to It than You Think [Proposes the SPACE framework (Satisfaction, Performance, Activity, Communication, Efficiency) for individual developer productivity. Explicit warning against reducing productivity to a single metric (SLOC, commit counts). Conceptual precedent for multi-dimensional measurement used in this thesis.]. *Communications of the ACM*, 64(11), 46–53. <https://doi.org/10.1145/3453120>
- Gloaguen, T., Mündler, N., Müller, M., Raychev, V., & Vechev, M. (2026). Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? [LLM-generated context files reduce success rate; developer-written marginally help (+4%); context files increase cost 20%+; agents follow instructions but no performance gain; AGENTBENCH benchmark (138 instances, 12 repos)]. *arXiv preprint arXiv:2602.11988*. <https://arxiv.org/abs/2602.11988>
- Gotel, O. C., & Finkelstein, A. C. (1994). An Analysis of the Requirements Traceability Problem. *Proceedings of IEEE International Conference on Requirements Engineering*, 94–101.
- Guo, J., Huang, S., Li, M., Huang, D., Chen, X., Zhang, R., Guo, Z., Yu, H., Yiu, S.-M., Lio, P., & Lam, K.-Y. (2025). A Comprehensive Survey on Benchmarks and Solutions in Software Engineering of LLM-Empowered Agentic System [Over 150 papers surveyed]. *arXiv preprint arXiv:2510.09721*. <https://arxiv.org/abs/2510.09721>
- Hassan, A. E., Li, H., Lin, D., Adams, B., Chen, T.-H., Kashiwa, Y., & Qiu, D. (2025). Agentic Software Engineering: Foundational Pillars and a Research Roadmap [SASE framework, BriefingScript, MentorScript, SE4H/SE4A duality]. *arXiv preprint arXiv:2509.06216*. <https://arxiv.org/abs/2509.06216>
- Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., ... Zaremba, W. (2021). Evaluating Large Language Models Trained on Code [Introduces HumanEval benchmark (164 hand-written Python problems) and Codex model. pass@k metric for functional correctness.]. *arXiv preprint arXiv:2107.03374*. <https://arxiv.org/abs/2107.03374>
- IEEE Computer Society. (2024). *Guide to the Software Engineering Body of Knowledge* (Version 4.0). <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- Inozemtseva, L., & Holmes, R. (2014). Coverage Is Not Strongly Correlated with Test Suite Effectiveness. *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 435–445. <https://doi.org/10.1145/2568225.2568271>
- ISO/IEC. (2023). *Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model* (tech. rep. No. ISO/IEC

- 25010:2023). International Organization for Standardization and International Electrotechnical Commission. Geneva. <https://www.iso.org/standard/78176.html>
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? [2294 tasks from real GitHub repositories, de facto benchmark for AI coding agents]. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2310.06770>
- Jin, H., Huang, L., Cai, H., Yan, J., Li, B., & Chen, H. (2024). From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future [Six SE areas covered: requirement engineering, code generation, autonomous decision-making, software design, test generation, software maintenance.]. *arXiv preprint arXiv:2408.02479*. <https://arxiv.org/abs/2408.02479>
- Jørgensen, M., & Shepperd, M. (2007). A Systematic Review of Software Development Cost Estimation Studies. *IEEE Transactions on Software Engineering*, 33(1), 33–53. <https://doi.org/10.1109/TSE.2007.256943>
- Kim, O. (2025). DETAIL Matters: Measuring the Impact of Prompt Specificity on Reasoning in Large Language Models [Emory University. Framework pro evaluaci vlivu specifčnosti promptu na reasoning. 30 úloh, GPT-4 a O3-mini]. *arXiv preprint arXiv:2512.02246*. <https://arxiv.org/abs/2512.02246>
- Kitchenham, B., & Pfleeger, S. L. (1996). Software Quality: The Elusive Target. *IEEE Software*, 13(1), 12–21. <https://doi.org/10.1109/52.476281>
- Lehman, M. M. (1980). Programs, Life Cycles, and Laws of Software Evolution. *Proceedings of the IEEE*, 68(9), 1060–1076. <https://doi.org/10.1109/PROC.1980.11805>
- Li, B., Latendresse, J., Abdalkareem, R., & Shang, W. (2026). Beyond Bug Fixes: Post-Merge Code Quality Issues in Agent-Generated Pull Requests [1210 merged agent PRs, SonarQube analysis. Merge success unreliable indicator of quality. Code smells, complexity, duplicated code persist.]. *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*. <https://arxiv.org/abs/2601.20109>
- Li, X., Chen, W., Liu, Y., Zheng, S., Chen, X., He, Y., Li, Y., et al. (2026). SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks [Curated Skills +16.2pp; self-generated Skills -1.3pp; 2–3 focused modules optimal; comprehensive documentation hurts (-2.9pp); smaller model + Skills can exceed larger model without; 86 tasks, 11 domains, 7308 trajectories]. *arXiv preprint arXiv:2602.12670*. <https://arxiv.org/abs/2602.12670>
- Lindenbauer, T., et al. (2025). The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management [Observation masking halves cost, matches LLM summarization; hybrid reduces cost 7–11% further]. *NeurIPS*.
- Liu, J., Wang, K., Chen, Y., Peng, X., Chen, Z., Zhang, L., & Lou, Y. (2024). Large Language Model-Based Agents for Software Engineering: A Survey [124 papers surveyed]. *arXiv preprint arXiv:2409.02977*. <https://arxiv.org/abs/2409.02977>
- Lulla, J. L., Mohsenimofidi, S., Galster, M., Zhang, J. M., Baltes, S., & Treude, C. (2026). On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents [124 PRs across 10 repos: -28% runtime, -20% output tokens with AGENTS.md; architectural

- info and coding conventions most effective]. *arXiv preprint arXiv:2601.20404*. <https://arxiv.org/abs/2601.20404>
- Mao, Y., He, J., & Chen, C. (2025). From Prompts to Templates: A Systematic Prompt Template Analysis for Real-world LLM Applications [Merged 4 frameworks (Google Cloud, Elavis Saravia, CRISPE, LangGPT); 7 component types; Directive 87% prevalence]. *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. <https://arxiv.org/abs/2504.02052>
- Mathews, N. S., & Nagappan, M. (2024). LLM-Based Test Generation as Bug Validation: Insights from Reproducing Real-World Bugs [68.1% generated test suites validate bugs instead of detecting them]. *Proceedings of the 1st ACM International Conference on AI-Powered Software (AIware)*. <https://doi.org/10.1145/3664646.3664766>
- McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering, SE-2*(4), 308–320. <https://doi.org/10.1109/TSE.1976.233837>
- McConnell, S. (2004). *Code Complete: A Practical Handbook of Software Construction* (2nd). Microsoft Press.
- McIntosh, S., Kamei, Y., Adams, B., & Hassan, A. E. (2016). An Empirical Study of the Impact of Modern Code Review Practices on Software Quality. *Empirical Software Engineering, 21*(5), 2146–2189. <https://doi.org/10.1007/s10664-015-9381-9>
- Merrill, M. A., et al. (2026). Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces [89 hard CLI tasks; Stanford and Laude Institute; frontier models below 65 %]. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2601.11868>
- METR. (2025). Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity [RCT: 16 developers, 246 tasks; AI tools slowed experienced developers by 19%; developers believed they were 24% faster]. *arXiv preprint arXiv:2507.09089*. <https://arxiv.org/abs/2507.09089>
- METR. (2026). Many SWE-bench-Passing PRs Would Not Be Merged into Main [296 patches reviewed by 4 maintainers from 3 SWE-bench repos. ~24pp gap between automated pass rate and maintainer merge decisions. Technical report.]. <https://metr.org/notes/2026-03-10-many-swe-bench-passing-prs-would-not-be-merged-into-main/>
- Min, S., Lyu, X., Holtzman, A., Artetxe, M., Lewis, M., Hajishirzi, H., & Zettlemoyer, L. (2022). Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? [Label space, input distribution a formát jsou klíčové pro ICL, ne správnost label-input mapování]. *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. <https://arxiv.org/abs/2202.12837>
- Nonaka, I., & Takeuchi, H. (1995). *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. Oxford University Press.
- Panickssery, A., Bowman, S. R., & Feng, S. (2024). LLM Evaluators Recognize and Favor Their Own Generations [Causal link: self-recognition → self-preference; linear correlation; fine-tuning pushes recognition to 90%+]. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2404.13076>
- Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., & Harman, M. (2019). Mutation Testing Advances: An Analysis and Survey [Mutation score stronger predictor of fault

- detection than structural coverage]. *Advances in Computers*, 112, 275–378. <https://doi.org/10.1016/bs.adcom.2018.03.015>
- Rafique, Y., & Mišić, V. B. (2013). The Effects of Test-Driven Development on External Quality and Productivity: A Meta-Analysis. *IEEE Transactions on Software Engineering*, 39(6), 835–856. <https://doi.org/10.1109/TSE.2012.28>
- Razavi, S. P., & Fard, F. H. (2025). What Did I Do Wrong? Quantifying LLMs’ Sensitivity and Consistency to Prompt Engineering [Up to 76 accuracy points difference from subtle formatting changes in few-shot settings]. *Proceedings of NAACL*. <https://arxiv.org/abs/2406.12334>
- Runeson, P., & Höst, M. (2009). Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>
- Scale AI. (2025). SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? [1865 human-verified tasks, adds explicit Requirements and Interface sections to issues]. *arXiv preprint arXiv:2509.16941*. <https://arxiv.org/abs/2509.16941>
- Shin, J., Tang, C., Mohati, T., Nayebi, M., Wang, S., & Hemmati, H. (2025). Prompt Engineering or Fine-Tuning: An Empirical Assessment of Large Language Models for Code. *Proceedings of the 22nd International Conference on Mining Software Repositories (MSR)*. <https://arxiv.org/abs/2310.10508>
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761*.
- Solis, C., & Wang, X. (2011). A Study of the Characteristics of Behaviour Driven Development. *37th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA)*, 383–387. <https://doi.org/10.1109/SEAA.2011.76>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.
- Verga, P., Hofstätter, S., Cer, D., & Thorne, J. (2024). Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models [Panel of LLM evaluators (PoLL) outperforms single GPT-4 judge; disjoint model families reduce intra-model bias; 7x cheaper]. *arXiv preprint arXiv:2404.18796*. <https://arxiv.org/abs/2404.18796>
- Watanabe, M., Li, H., Kashiwa, Y., Reid, B., Iida, H., & Hassan, A. E. (2025). On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub [Under review. 567 Claude Code PRs, 157 projects, 83.8% acceptance rate]. *ACM Transactions on Software Engineering and Methodology*.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models [Google Research. Few-shot CoT exempláře zlepšují reasoning na aritmetických, commonsense a symbolických úlohách]. *Advances in Neural Information Processing Systems (NeurIPS)*, 35. <https://arxiv.org/abs/2201.11903>
- Yang, J., Jimenez, C. E., Wettig, A., Liber, K., Narasimhan, K., & Press, O. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering [ACI

- ablation: +10.7pp over baseline shell; context management and error guardrails largest gains]. *NeurIPS*.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*.
- Ye, S., Sun, Z., Wang, G., Guo, L., Liang, Q., Li, Z., & Liu, Y. (2025). Prompt Alchemy: Automatic Prompt Refinement for Enhancing Code Generation. *arXiv preprint arXiv:2503.11085*. <https://arxiv.org/abs/2503.11085>
- Yin, R. K. (2018). *Case Study Research and Applications: Design and Methods* (6th ed.) [Embedded single-case design; analytic generalization (to theory, not population)]. SAGE Publications.
- Yin, Z., Wang, J., Li, J., Shi, D., Wei, Y., Yue, Y., Liu, Z., Xia, X., & Lo, D. (2025). A Comprehensive Empirical Evaluation of Agent Frameworks on Code-centric Software Engineering Tasks. *arXiv preprint arXiv:2511.00872*.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena [Foundational LLM-as-judge paper; position bias, verbosity bias, self-enhancement bias; GPT-4 >80% human agreement]. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2306.05685>
- Zhou, Q., Zhang, J., Wang, H., Hao, R., Wang, J., Han, M., Yang, Y., Wu, S., Pan, F., Fan, L., Tu, D., & Zhang, Z. (2026). FeatureBench: Benchmarking Agentic Coding for Complex Feature Development [Agents struggle when scope exceeds single-issue; feature-level tasks significantly harder]. *arXiv preprint arXiv:2602.10975*. <https://arxiv.org/abs/2602.10975>
- Zi, Y., Menon, H., & Guha, A. (2025). More Than a Score: Probing the Impact of Prompt Specificity on LLM Code Generation [PartialOrderEval framework. Partial order of prompts from minimal to maximally detailed. HumanEval + ParEval]. *arXiv preprint arXiv:2508.03678*. <https://arxiv.org/abs/2508.03678>

Přílohy

A. Využití nástrojů umělé inteligence

Při přípravě práce byl využit nástroj Claude Code (Anthropic) jako AI asistent v následujících oblastech:

Psaní textu práce. Nástroj byl využit pro drafting a úpravu textových částí (kapitoly 1–5), kontrolu konzistence argumentace a LaTeX formátování. Autor formuloval záměr a strukturu každé sekce. AI asistent navrhoval formulace, které autor revidoval a upravoval. Metodika, argumentace a finální znění jsou autorovy vlastní.

Vyhledávání a shrnutí literatury. Nástroj byl využit k identifikaci relevantních zdrojů a shrnutí obsahu zdrojů. Autor ověřoval relevanci a správnost každého zdroje před zařazením do textu. Citace odkazují výhradně na zdroje přečtené a posouzené autorem.

Diagnostická analýza ve studii. V kroku Diagnóza iteračního cyklu (sekce 3.1.1) byl nástroj využit jako analytický asistent. Výzkumník popisoval naměřené výsledky, AI asistent navrhoval možné příčiny selhání a opravy instrukcí na základě literatury. Finální rozhodnutí o každé změně `AGENTS.md` bylo vždy na autorovi.

Implementace měřicí infrastruktury. Experimentální skripty (`new-run.ts`, `analyze-run.ts`, `judge.ts`) byly implementovány s asistencí AI nástroje na základě autorova návrhu specifikace metrik. Autor navrhl architekturu, definoval požadované výstupy a provedl validaci správnosti měření.

Claude Code nepouštěl experimentální běhy ani neprováděl finální interpretaci výsledků a nerozhodoval o směru výzkumu. Pro hodnocení části metrik (P6, P7, P8, Q4, Q8) byla ve studii samostatně použita metoda LLM-as-judge s modelem GLM-5. Její role, omezení a ověřitelnost jsou popsány v sekci 3.2. Za celý obsah práce nese plnou odpovědnost autor.

B. Doplnující materiály k případové studii

Tato příloha odkazuje na doplňující materiály uložené ve veřejném repozitáři práce ve stavu označeném tagem `thesis-final`. Strukturu release stavu popisuje kořenový soubor `README.md`. Jednotlivé adresáře níže vedou na podklady použité pro ověření metodiky, běhů a naměřených hodnot.

Tabulka 24: Doplnující materiály k případové studii

Materiál	Odkaz
Struktura release stavu	<code>README.md</code>
Metodické a technické podklady	<code>experiments/infra</code>
Běhy agenta	<code>experiments/runs</code>
Referenční implementace	<code>experiments/reference</code>
Repozitář práce	<code>github.com/7onc3k/Bakalarka/tree/thesis-final</code>